- 一、CSS定义
- 二、CSS引入方式
- CSS特性
  - ①继承性
  - ②层叠性
  - ③优先级
    - a.优先级-叠加规则
- 三、选择器
- 1.基础选择器
  - ①标签选择器
  - ②类选择器
  - ③id选择器
  - 通配符选择器
- 2.复合选择器
  - ①后代选择器
  - ②子代选择器
  - ③并集选择器
  - ④交集选择器 - 了解
- 3.伪类选择器
  - 伪类-超链接（拓展）
- 4.结构伪类选择器
  - :nth-child(公式)
- 5.伪元素选择器
- 四、画盒子
- 1.盒子模型--组成
- ①盒子模型-边框线
  - 设置单方向边框线
- ②盒子模型-内边距
  - 盒子模型-内边距-多值写法
- 2.盒子模型-尺寸计算
- 3.盒子模型-外边距
  - 盒子模型-版心居中
- 4.清除默认样式
- 5.盒子模型-元素溢出
- 6.外边距问题
  - ①合并现象
  - ②塌陷问题
- 7.行内元素-内外边距问题
- 8.盒子模型-圆角
  - ①多值写法：

- ②常用圆角
- 9.盒子模型-阴影（拓展）
- 10.盒子结构化布局
  - ①标准流
  - ②浮动
    - A.浮动应用--产品区域布局
      - HTML标签
      - CSS样式
    - B.浮动的影响
    - C.清除浮动影响的方法
      - ①额外标签法
      - ②单伪元素法
      - ③双伪元素法
      - ④overfow法
  - ③Flex布局
    - 1.Flex组成
    - 2.主轴对齐方式
    - 3.侧轴对齐方式
    - 4.修改主轴方向
    - 5.弹性伸缩比
    - 6.弹性盒子换行
    - 7.行内对齐方式
- 五、背景属性
  - 1.背景色
  - 2.背景图
    - A.背景图平铺方式
    - 背景图位置
    - B.背景图缩放
    - C.背景图固定
    - D.背景复合属性
- 六、显示模式
  - 1.块级元素
  - 2.行内元素
  - 3.行内块元素
  - 4.转换显示模式
- 七、CSS高级
  - 01-定位
    - 相对定位
    - 绝对定位
    - 定位居中

- 固定定位
- 堆叠层级z-index
- 02-高级技巧
  - CSS精灵
  - 案例-京东服务
    - HTML结构
    - CSS样式
  - 字体图标
    - 下载字体
    - 使用字体
    - 上传矢量图
- 03-CSS修饰属性
  - 垂直对齐方式
  - 过渡
  - 透明度opacity
  - 光标类型cursor
- 04-综合案例-轮播图
  - 图片效果
    - HTML结构
    - CSS样式
  - 箭头
    - HTML结构
    - CSS样式
  - 圆点
    - HTML结构
    - CSS样式

八、移动WEB

- 01-平面转换
  - 简介
  - 平移
  - 定位居中
  - 案例-双开门
  - 旋转
  - 转换原点
  - 案例-时钟
  - 多重转换
  - 缩放
  - 案例-播放特效
  - 倾斜
- 02-渐变

- 线性渐变
- 案例-产品展示
- 径向渐变
- 03-综合案例
  - 导航-频道
    - 箭头旋转
    - 频道列表
  - 渐变按钮
    - 搜索按钮
    - 登录按钮
    - 客户端按钮
  - 轮播图
  - 猜你喜欢
- 04-空间转换
  - 空间转换简介
  - 平移
  - 视距
  - 空间-旋转
  - 左手法则
  - rotate3d-了解
  - 立体呈现
  - 案例-3d导航
  - 空间缩放
- 05-动画
  - 动画实现步骤
  - animation复合属性
  - animation拆分写法
  - 案例-走马灯
  - 精灵动画
  - 多组动画
- 06-综合案例-全名出游
  - 背景
  - 云彩位置和动画
  - 文字动画
- 07-移动 Web 基础
  - 谷歌模拟器
  - 屏幕分辨率
  - 视口
  - 二倍图
  - 适配方案

- 08-rem
  - 简介
  - 媒体查询
  - rem 布局
  - flexible.js
  - rem 移动适配
- 09-less
  - 注释
  - 运算
  - 嵌套
  - 变量
  - 导入
  - 导出
  - 禁止导出
- 10-综合案例-极速问诊
  - 准备工作
  - 头部布局
  - 头部内容
  - banner 区域
  - 问诊类型布局
  - 问诊类型内容
- 11-vw适配方案
  - vw和vh基本使用
  - vw布局
  - vh问题
- 12-综合案例-酷我音乐
  - 准备工作
  - 头部布局
  - 头部内容
  - 搜索区域
  - banner 区域
  - 标题公共样式
  - 排行榜内容
  - 推荐歌单布局
  - 推荐歌单内容
  - 下载区域
  - 头部固定
  - 响应式布局方案
- 13-媒体查询
  - 基本写法

- 书写顺序
- 案例-左侧隐藏
- 媒体查询-完整写法
  - 关键词 / 逻辑操作符
  - 媒体类型
  - 媒体特性
- 媒体查询-外部CSS
- 14-Bootstrap
  - 简介
  - 使用步骤
    - 下载
    - 使用
  - 栅格系统
  - 全局样式
    - 按钮
    - 表格
  - 组件(Components)
  - 字体图标
    - 下载
    - 使用
- 15-综合案例-alloyTeam
  - 准备工作
  - 头部导航
  - 轮播图图片
  - 轮播图响应式
  - 开源项目响应式
  - 开源项目内容布局

九、文字控制属性

- 控制文字的属性名
  1. `font-size`：字体大小，单位`px`
  2. `font-weight`:字体粗细
  3. `font-style`：字体倾斜
  4. `line-height`:行高
  5. `font-family`:字体族，如宋体
  6. `font`:字体复合属性
  7. `text-indent`：文本缩进，如首行缩进
  8. `text-align`:文本对齐
  9. `text-decoration`:修饰线
  10. `color`：颜色

十、Emmet写法

一、CSS定义

层叠样式表，是一种样式表语言，用来描述HTML文档的呈现（美化内容
书写位置：`title`标签下方添加`style`双标签，`style`标签里面书写CSS代码。

```
<title>CSS初体验</title>
<style>
        /*选择器{CSS属性}*/
        p{
        color:red;

        }
 </style>

 <p>体验CSS</p>
```

二、CSS引入方式

1.内部样式表：学习使用
CSS 代码写在`style`标签里面

2.外部样式表：开发使用
a.CSS代码写在单独的CSS文件中（.css）
b.在HTML使用`link`标签引入
`<link rel="stylesheet" href="./my.css">`
my.css文件写：`p{color: red;}`
写法：`选择器{CSS属性}`

3.行内样式：配合JavaScript使用
a.CSS写在标签的`style`属性值里
`<div style="color:red;front-size:20px">这是div标签</div>` 写法：style="CSS属性值"`

CSS特性

①继承性

子级默认继承父级的文字控制属性。

```
<style>
    body{
        font-size:30px;
        color:red;
        font-weight:700;
    }
</style>

<body>
    <div>div标签</div>
    <span>123</span>
    <p>哈喽</p>

    <!--如果标签自己有样式就生效自己的样式，就不继承父级的样式-->
    <a herf="#">a标签</a>
    <h1>h1标签</h1>
</body>
```

②层叠性

特点：
a.相同的属性会覆盖：后面的CSS属性覆盖前面的CSS属性
b.不同的属性会叠加：不同的CSS属性都生效

```
<style>
/*覆盖，后面的覆盖前面的；叠加*/
    div{
        color:red;
        font-weight:700;
    }
    div{
        color:green;
        font-size:30px;
    }
</style>

<body>
    <div>div标签</div>
</body>
```

③优先级

优先级也叫权重，当一个标签使用了多种选择器时，基于不同种类的选择器的匹配机制。
规则：选择器优先级高的样式生效
公式：通配符选择器<标签选择器<类选择器< id选择器<行内样式<！important
选中标签的范围越大，优先级越低

```
<style>
/*技巧：选择器选中标签的范围  越大，优先级/权重  越低*/
    /*!important提权功能，提高权重/优先级到 最高，慎用*/
    *{
        color:red !important;
    }
    div{
        color:green;
    }
    .one{
        color:blue;
    }
    #test{
        color:orange;
    }

</style>

<body>
    <div class="one" id="test" style="color:purple">div标签</div>
</body>
```

a.优先级-叠加规则

叠加计算：如果是复合选择器，则需要权重叠加计算。
公式：（每一级之间不存在进位）
（行内样式，id选择器个数，类选择器个数，标签选择器个数）
规则：
a.从左向右依次比较选个数，同一级个数多的优先级高。如果个数相同，则向后比较
b.`!important`权重最高
c.继承权重最低

三、选择器

作用：查找标签，设置样式。

1.基础选择器

①标签选择器

使用标签名作为选择器 → 选中同名标签设置相同的样式
例如：p,h1,div,a,img....

②类选择器

作用：查找标签，差异化设置标签的显示结果。
步骤：
1.定义类选择器 → .类名
2.使用类选择器 → 标签添加`class="类名"`
3.一个标签可以使用多个类名，类名之间用空格隔开

开发习惯：类名见名知意，多个单词可以用-连接，例如`news-hd`

```
<style>
    .red{
        color:red;
    }
    .size{
        f0nt-size:30px;
    }
</style>

<body>
<!--一个标签可以使用多个类名，class属性值写多个类名，类名用空格隔开-->
    <p class="red size">哈喽</p>
</body>
```

③id选择器

作用：查找标签，差异化设置标签的显示结果。
场景：id选择器一般配合JavaScript使用，很少用来设置CSS样式
步骤：
1.定义id选择器 → `#id`名
2.使用id选择器 → 标签添加id="id名"
规则：同一个id选择器在一个页面中只能使用一次

```
<style>
/*定义id选择器*/
    #red{
        color:red;
    }

</style>

<body>
<!--使用id选择器-->
    <p id="red">哈喽</p>
</body>
```

通配符选择器

作用：查找页面所有标签，设置相同样式。
通配符选择器*，不需要调用，浏览器自动查找页面所有标签，设置相同的样式
`*{color:red;}`
场景：开发初期清除默认样式

2.复合选择器

定义：由两个或多个基础选择器，通过不同方式组合而成。
作用：更准确、更高效的选择目标元素（标签）。

①后代选择器

作用：选中某元素的后代元素。能选中所有后代，包含儿子，孙子，重孙子...
写法：父选择器 子选择器（CSS属性），父子选择器之间用空格隔开。

```
<style>
    div span{
        color: red;
        height:100px;
        background-color:skyblue;
        line-height:100px;
        }
</style>

<body>
    <span> span标签</span>
    <div>
    <span>这是div的儿子</span>
    <p>
    <span>这是div的孙子</span>
    </p>
    </div>
</body>
```

②子代选择器

作用：选中某元素的子代元素（最近的子级）。
写法：父选择器>子选择器（CSS属性），父子选择器之间用`>`隔开。

```
<style>
    div>span{
        color: red;
        }
</style>

<body>
    <span> span标签</span>
    <div>
    <span>这是div的儿子</span>
    <p>
    <span>这是div的孙子</span>
    </p>
    </div>
</body>
```

③并集选择器

作用：选中多组标签设置相同的样式。
写法：选择器1,选择器2,...,选择器N（CSS属性），选择器之间用`,`隔开。

```
<style>
    div>,
    span,
    p{
        color：red;
        }
</style>

<body>
    <span> span标签</span>
    <div>这是div</div>
    <p>标签</p>
</body>
```

④交集选择器 - 了解

作用：选中同时满足多个条件的元素。
写法：选择器1选择器2（CSS属性），选择器之间连写，没有任何符号。

```
<style>
    p.box{
        color：red;
        }
</style>

<body>
    <span> span标签</span>
    <div class="box">这是div</div>
    <p class="box">标签</p>
</body>
```

3.伪类选择器

定义：伪类表示元素状态，选中元素的某个状态设置样式。
鼠标悬停状态：选择器：hover（CSS属性）

```
<style>
    span：hover{
        color：red;
        }
</style>

<body>
    <span> span标签</span>
    <div class="box">这是div</div>
    <p class="box">标签</p>
</body>
```

伪类-超链接（拓展）

超链接一共有四个状态
`:link`:访问前
`:visited`： 访问后
`:hover`： 鼠标悬停
`:active`： 点击时（激活）
提示：如果要给超链接设置以上四个状态，需要按照LVHA的顺序写

```
<style>
    a:{
        color:red;
    }
    a：hover{
        color：green;
        }
</style>

<body>
    <a>a标签，测试伪类</a>
</body>
```

4.结构伪类选择器

作用：根据元素的结构关系查找元素
1. `E:first-child`： 查找第一个E元素
2. `E:last-child`:查找最后一个E元素
3. `E:nth-child(N)`： 查找第N个E元素（第一个元素N值为1）

```
<style>
    li:first-child {
            background-color: green;
        }

    li:last-child {
            background-color: red;
        }

    li:nth-child(3) {
            background-color: burlywood;
        }
</style>

<body>
    <ul>
        <li>li 1</li>
        <li>li 2</li>
        <li>li 3</li>
        <li>li 4</li>
        <li>li 5</li>
        <li>li 6</li>
        <li>li 7</li>
        <li>li 8</li>
    </ul>
</body>
```

:nth-child(公式)

作用：根据元素的结构关系查找多个元素
公式：

1. `2n`： 偶数标签
2. `2n+1;2n-1`： 奇数标签
3. `5n`： 找到5的倍数标签
4. `n+5`： 找到第五个以后的标签
6. `-n+5`： 找到第五个以前的标签

```
<style>
    /*li:nth-child(2n) {
            background-color: burlywood;
        }*/
    /*li:nth-child(2n-1) {
            background-color: burlywood;
        }*/
    /*li:nth-child(5n) {
            background-color: burlywood;
        }*/
    /*li:nth-child(n+5) {
            background-color: burlywood;
        }*/
    li:nth-child(-n+5) {
            background-color: burlywood;
        }

</style>

<body>
    <ul>
        <li>li 1</li>
        <li>li 2</li>
        <li>li 3</li>
        <li>li 4</li>
        <li>li 5</li>
        <li>li 6</li>
        <li>li 7</li>
        <li>li 8</li>
        <li>li 9</li>
        <li>li 10</li>
    </ul>
</body>
```

5.伪元素选择器

作用：创建虚拟元素（伪元素），用来摆放装饰性的内容

选择器：

1. `E::before`:在E元素里面最前面添加一个伪元素
2. `E::after`： 在E元素里面最后面添加一个伪元素

注意点：

1.必须设置content:" "属性，用来设置伪元素的内容，如果没有内容，则引号留空即可

2.伪元素默认是行内显示模式
3.权重和标签选择器相同

```
<style>
    div::before {
            content: "老鼠";
        }
/*必须设置content属性，没有content，伪元素不生效*/
    div::after {
            content: "大米";
        }

</style>

<body>
    <div>爱</div>
</body>
```

四、画盒子

目标：使用合适的选择器画盒子
新属性：
1. `width`:宽度
2. `height`:高度
3. `background-color`：背景色

```
<style>
    .red{
        width:100px;
        height:100px;
        background-color:brown;
    }
    .orange{
        width:200px;
        height:200px;
        background-color:orange;
    }

</style>

<body>
    <div class="red">div1</div>
    <div class="orange">div2</div>
</body>
```

1.盒子模型--组成

作用：布局网页，摆放盒子和内容。
盒子模型重要组成部分：
1. `width & height`：内容区域
2. `padding`：内边距（出现在内容与盒子边缘之间
3. `border`：边框线
4. `margin`：外边距（出现在盒子外面）

```
<style>
        div {
            width: 100px;
            height: 100px;
            background-color: pink;

            /*内容与盒子边缘之间*/
            padding: 20px;

            border: 1px solid #000;

            /*出现在盒子外面，拉开两个盒子之间的距离*/
            margin: 50px;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

①盒子模型-边框线

属性名：`border`（bd）
属性值：边框线的粗细 线条样式 颜色（不区分顺序）
a.`solid`：实线
b.`dashed`：虚线
c.`dotted`：点线

```
<style>
        div {
            width: 100px;
            height: 100px;
            background-color: pink;

           /* border: 1px solid #000;*/

           /* border: 1px dashed #e6590c;*/

            border: 3px dotted green;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

设置单方向边框线

属性名：`border-方位名词`(bd+方位名词首字母，例如，bdl)
属性值：边框线粗细 线条样式 颜色（不区分顺序）

```
<style>
        div {
            width: 100px;
            height: 100px;
            background-color: pink;

           border-top: 1px solid #000;

           border-right: 1px dashed #e6590c;

            border-left: 2px dashed #8edf0d;

            border-bottom: 3px dotted green;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

②盒子模型-内边距

作用：设置内容与盒子边缘之间的距离
属性名：`padding/padding-方位名词`

```
<style>
        div {
            width: 100px;
            height: 100px;
            background-color: pink;

            /*padding:20px;*/

            padding-top: 20px;
            padding-right: 30px;
            padding-left: 10px;
            padding-bottom: 80px;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

盒子模型-内边距-多值写法

1. `padding:10px`：四个方向内边距均为10px
2. `padding:10px 20px 40px 80px`：上：10px；右：20px；下：40px；左：80px
3. `padding:10px 40px 80px`：上：10px；左右：40px；下：80px
4. `padding:10px 80px`：上下：10px；左右：80px

记忆方法：从上开始顺时针转一圈，如果当前方向没有数值，取值跟对面一样

2.盒子模型-尺寸计算

默认情况：盒子尺寸=内容尺寸+border尺寸+内边距尺寸
结论：给盒子加border/padding会撑大盒子

解决：
a.手动做减法，减掉border/padding的尺寸
b.内减模式：`box-sizing:border-box`

```
<style>
        div {
            width: 100px;
           height: 100px;

            /*手动减法*/
            /* width:60px;
            height:60px;*/
            background-color: pink;

            padding: 20px;
            /*内减模式*/
            box-sizing:border-box;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

3.盒子模型-外边距

作用：拉开两个盒子之间的距离
属性名：`margin/margin-方位名词`
多值写法跟`padding`一样

```
<style>
        div {
           width: 100px;
           height: 100px;

        /*外边距不会撑大盒子*/
            margin:50px;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

盒子模型-版心居中

`margin:0 auto;`
版心居中要求：盒子要有宽度

```
<style>
        div {
            width: 100px;
            height: 100px;
            background-color: pink;

            /*版心居中要求：盒子要有宽度*/
            margin:0 auto;
        }
</style>

<body>
    <div>这是div标签1</div>
</body>
```

4.清除默认样式

清除标签默认的样式，如：默认的内外边距。

```
<style>
    *{
        margin:0;
        padding:0;
    }

    /*去掉列表的项目符号*/
    li{
        list-style:none;
    }
</style>

<body>
    <h1>标题</h1>
    <p>ppppp</p>
    <ul>
        <li>li</li>
        <li></li>
    </ul>
</body>
```

5.盒子模型-元素溢出

作用：控制溢出元素的内容的显示方式
属性名：`overflow`
属性值：
a.`hidden`：溢出隐藏
b.`scroll`:溢出滚动（无论是否溢出，都显示滚动条位置）
c.`auto`：溢出滚动（溢出才会显示滚动条位置）

```
<style>
        div {
            width: 100px;
           height: 100px;

            overflow: hidden;
           /* overflow: scroll;*/
            /*overflow: auto;*/
        }
</style>

<body>
    <div>这是div标签1这是div标签1这是div标签1这是div标签1这是div标签1这是div标签1这是div标签1
    这是div标签1这是div标签1这是div标签1这是div标签1
    这是div标签1这是div标签1这是div标签1这是div标签1</div>
</body>
```

6.外边距问题

①合并现象

场景：垂直排列的兄弟元素，上下margin会合并
现象：取两个`margin`中的较大值生效

```
 <style>
        .one{
            width:100px;
            height:100px;
            background-color:red;

            margin-bottom:20px;
        }
        .two{
            width:100px;
            height:100px;
            background-color:green;

            margin-top:50px;
        }
    </style>

</head>

<body>
    <div class="one">这是div标签2</div>
    <div class="two">这是div标签3</div>
</body>
```

②塌陷问题

场景：父子级的标签，子级的添加上外边距会产生塌陷问题
现象：导致父级一起向下移动

```
 <style>
        .father{
            width:100px;
            height:100px;
            background-color:red;
        }
        .son{
            width:100px;
            height:100px;
            background-color:green;

            margin-top:50px;
        }
    </style>

</head>

<body>
    <div class="father">
    <div class="son">son</div>
    </div>

</body>
```

解决方法：

1.取消子级margin，父级设置padding

2.父级设置overflow：hidden

3.父级设置border-top

```
    <style>
        .father {
            width: 300px;
            height: 300px;
            background-color: red;

            /*padding-top: 50px;
            box-sizing: border-box;*/

            /*溢出隐藏*/
            /*overflow: hidden;*/
            border-top: 1px solid #000;
        }

        .son {
            width: 100px;
            height: 100px;
            background-color: green;

            margin-top: 50px;
        }
    </style>

</head>

<body>
    <div class="father">
        <div class="son">son</div>
    </div>

</body>
```

7.行内元素-内外边距问题

场景：行内元素添加margin和padding，无法改变元素垂直位置
解决办法：给行内元素添加line-height
可以改变垂直的位置

```
 <style>
        span {
            margin :50px;
            padding :20px;
            line-height: 100px;
        }
    </style>

</head>

<body>
    <span>span </span>
    <span>span </span>

</body>
```

8.盒子模型-圆角

作用：设置元素的外边框为圆角
属性名：`border-radius`
属性值：`数字+px/百分比`
提示：属性值是圆角的半径

```
<style>
        div {
            margin: 50px auto;
            width: 200px;
            height: 200px;
            background-color: red;

            border-radius: 20px;

        }
    </style>

</head>

<body>
    <div></div>

</body>
```

①多值写法：

1.一值：`border-radius:10px`：每个角均为10px
2.四值：`border-radius:10px 20px 40px 80px`：左上：10px；右上：20px；右下：40px；左下：80px
3.三值：`border-radius:10px 40px 80px`：左上：10px；右上+左下：40px；右下：80px
4.两值：`border-radius:10px 80px`：左上+右下：10px；右上+左下：80px
记忆：从左上角顺时针赋值：没有取值的角与对角取值相同

②常用圆角

1.正圆形状：给正方形盒子设置圆角属性值为宽高的一半/50%，如头像



2.胶囊形状：给长方形盒子设置圆角属性值为盒子高度的一半



```
    <style>
        img {
            width: 200px;
            height: 200px;

            /*border-radius:100px;*/

            /*最大值是50%，超过50%没有效果*/
            border-radius: 50%;
        }

        div {
            width: 200px;
            height: 80px;
            background-color: red;
            border-radius: 40px;
        }
    </style>

</head>

<body>
    <img src="./111.png" alt="">
    <div></div>

</body>
```

9.盒子模型-阴影（拓展）

作用：给元素设置阴影效果
属性名：`box-shadow`
属性值：x轴偏移量 y轴偏移量 模糊半径 扩散半径 颜色 内外阴影
注意：
1.x轴偏移量和y轴偏移量 必须写
2.默认是外阴影，内阴影需要添加`inset`

```
    <style>
        div {
            margin: 50px auto;
            width: 200px;
            height: 80px;
            background-color: red;
            border-radius: 40px;

            box-shadow: 2px 5px 10px 1px #000 inset;
        }
    </style>

</head>

<body>
    <div></div>

</body>
```

10.盒子结构化布局

> 目标：熟练使用 Flex 完成结构化布局

①标准流

标准流也叫文档流，指的是标签在页面中默认的排布规则，例如：块元素如`<div>`独占一行，行内元素如`<span>`可以一行显示多个。

②浮动

作用：让块元素水平排列。本质作用是实现图文混排的效果
属性名：`float`
属性值:
a.`left`：左对齐
b.`right`：右对齐

特点：
1.浮动后的盒子顶对齐
2.浮动后的盒子具备行内块特点
3.浮动后的盒子脱标，不占用标准流的位置
4.父级宽度不够，浮动的子级会换行

```
<style>
  /* 特点：顶对齐；具备行内块显示模式特点；浮动的盒子会脱*/
  .one {
    width: 100px;
    height: 100px;
    background-color: brown;

    float: left;
  }

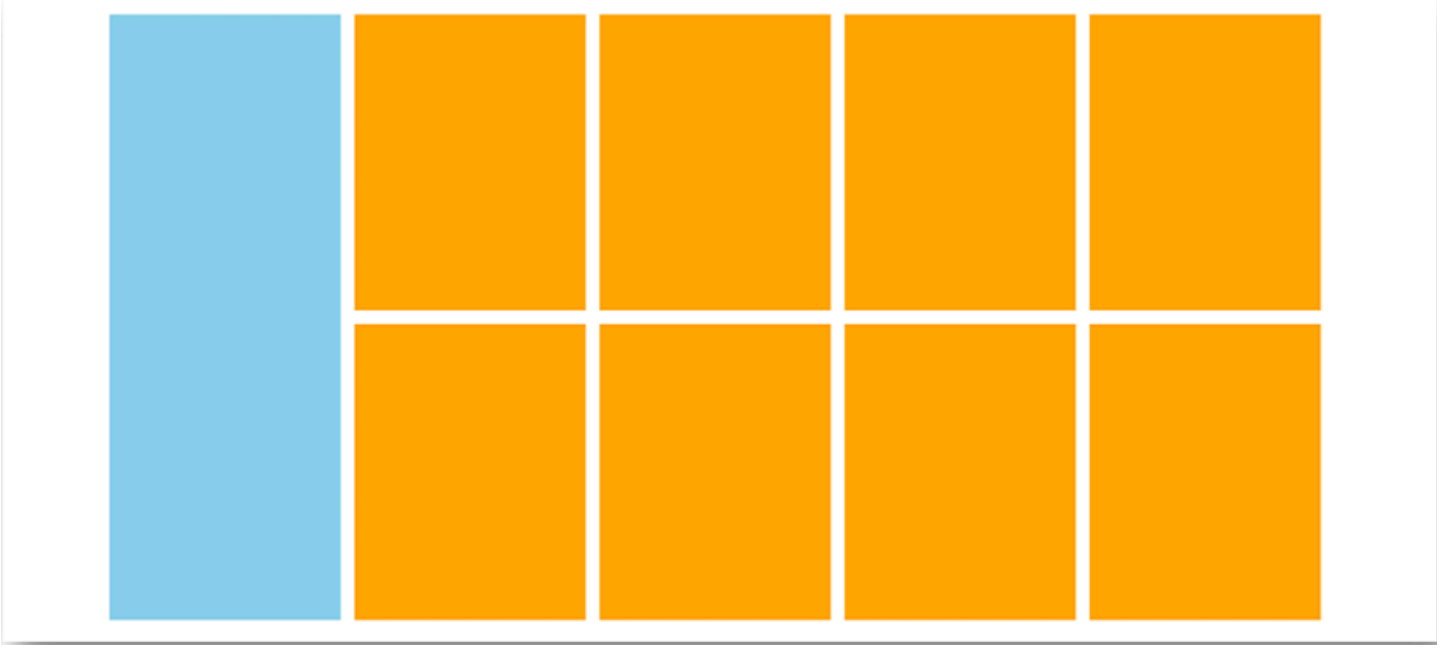
  .two {
    width: 200px;
    height: 200px;
    background-color: orange;

    /* float: left; */

    float: right;
  }
</style>

<div class="one">one</div>
<div class="two">two</div>
```

A.浮动应用--产品区域布局



HTML标签

```
<!-- 版心：左右，右面：8个产品 → 8个 li -->
<div class="product">
  <div class="left"></div>
  <div class="right">
    <ul>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
      <li></li>
    </ul>
  </div>
</div>
```

CSS样式

```
<style>
  * {
    margin: 0;
    padding: 0;
  }

  li {
    list-style: none;
  }

  .product {
    margin: 50px auto;
    width: 1226px;
    height: 628px;
    background-color: pink;
  }

  .left {
    float: left;
    width: 234px;
    height: 628px;
    background-color: skyblue;
  }

  .right {
    float: right;
    width: 978px;
    height: 628px;
    background-color: brown;
  }

  .right li {
    float: left;
    margin-right: 14px;
    margin-bottom: 14px;
    width: 234px;
    height: 300px;
    background-color: orange;
  }

  /* 第四个li和第八个li 去掉右侧的margin */
  .right li:nth-child(4n) {
    margin-right: 0;
  }
```

```
  /* 细节：如果父级宽度不够，浮动的盒子会掉下来 */
</style>
```

B.浮动的影响

场景：浮动元素会脱标，如果父级没有高度，子级无法撑开父级高度（可能导致页面布局错乱）



```
<style>
  .top {
    margin: 10px auto;
    width: 1200px;
    /* height: 300px; */
    background-color: pink;
  }

  .left {
    float: left;
    width: 200px;
    height: 300px;
    background-color: skyblue;
  }

  .right {
    float: right;
    width: 950px;
    height: 300px;
    background-color: orange;
  }

  .bottom {
    height: 100px;
    background-color: brown;
  }

</style>

<body>
<div class="top">
  <div class="left"></div>
  <div class="right"></div>
</div>
<div class="bottom"></div>
</body>
```

解决方法：清除浮动（清除浮动带来的影响）

C.清除浮动影响的方法



①额外标签法

在**父元素内容的最后**添加一个**块级**元素，设置 CSS 属性 **clear: both**

```
<style>
.clearfix {
  clear: both;
}
</style>

<div class="father">
  <div class="left"></div>
  <div class="right"></div>
  <div class="clearfix"></div>
</div>
 <div class="bottom"></div>
```

②单伪元素法

1. 准备 after 伪元素

```
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

2. 父级使用 clearfix 类

```
<div class="father clearfix">
    <div class="left"></div>
    <div class="right"></div>
    </div>
 <div class="bottom"></div>
```

③双伪元素法

1. 准备 after 和 before 伪元素

```
/* before 解决外边距塌陷问题 */
/* 双伪元素法 */
.clearfix::before,
.clearfix::after {
  content: "";
  display: table;
}

/* after 清除浮动 */
.clearfix::after {
  clear: both;
}
```

2. 父级使用 clearfix 类

```
<div class="father clearfix">
    <div class="left"></div>
    <div class="right"></div>
</div>
<div class="bottom"></div>
```

④overfow法

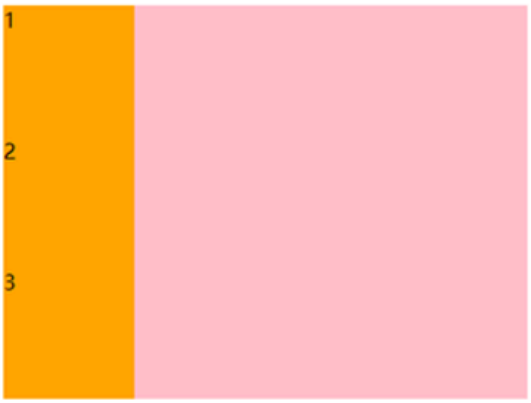
父元素添加CSS属性overflow:hidden

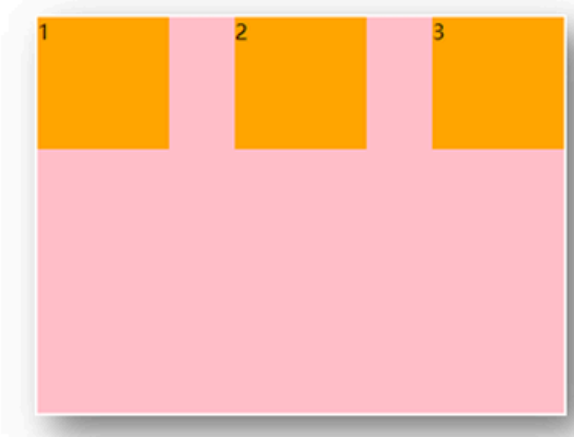
```
.father {
  margin: 10px auto;
  width: 1200px;
  /* height: 300px; */
  background-color: pink;

  overflow: hidden;
}
```

③Flex布局

Flex 布局也叫**弹性布局**，是浏览器**提倡的布局模型**，非常适合**结构化**布局，提供了强大的空间分布和对齐能力。
Flex 模型不会产生浮动布局中脱标现象，布局网页更简单、更灵活。



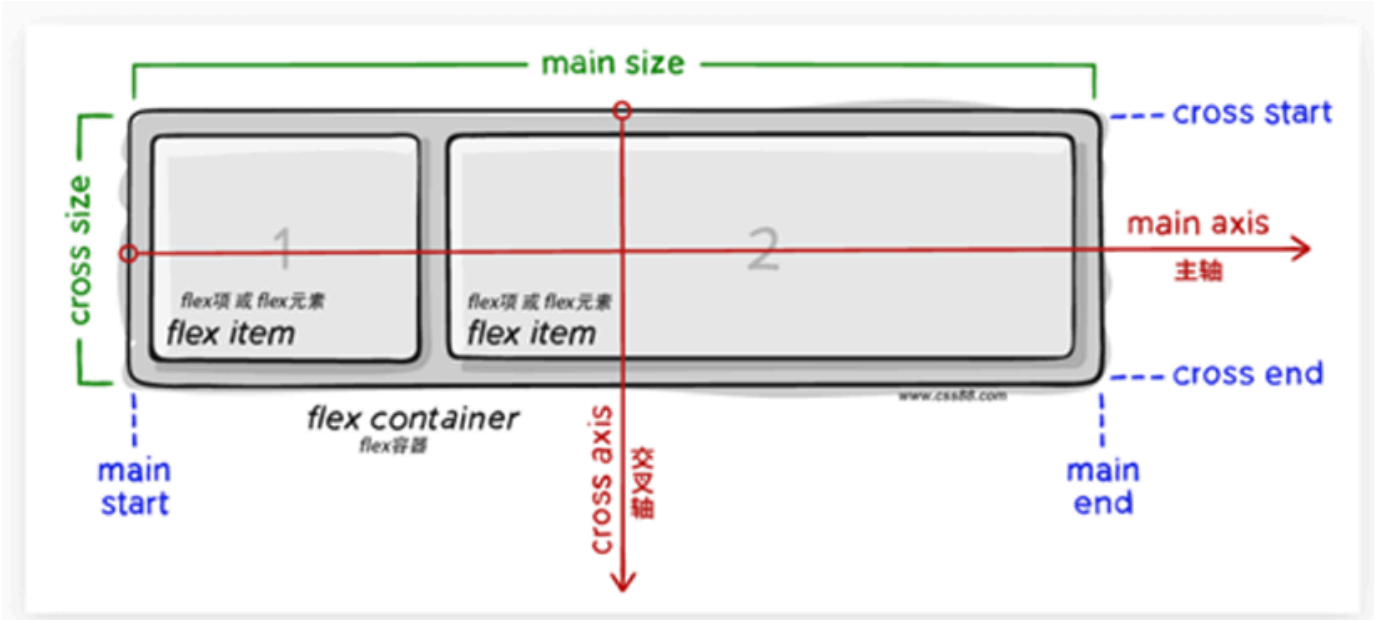


1.Flex组成

设置方式：给**父**元素设置 **display: flex**，子元素可以自动挤压或拉伸

组成部分：

- 弹性容器
- 弹性盒子
- 主轴：默认在**水平**方向
- 侧轴 / 交叉轴：默认在**垂直**方向



```
<style>
        /*弹性容器*/
        .box {
            height: 300px;
            border: 1px solid black;

            display: flex;
        }
        /*弹性盒子：默认沿着主轴方向排列*/
        .box div {
            width: 200px;
            height: 100px;
            background-color: pink;
        }
    </style>
<body>
    <div class="box">
        <div>1</div>
        <div>2</div>
        <div>3</div>
    </div>
    </body>
```

2.主轴对齐方式

属性名：**justify-content**

属性值	效果
flex-start	默认值，弹性盒子从**起点**开始依次排列
flex-end	弹性盒子从**终点**开始依次排列
center	弹性盒子沿主轴**居中**排列
space-between	弹性盒子沿主轴均匀排列，空白间距均分在弹性盒子**之间**
space-around	弹性盒子沿主轴均匀排列，空白间距均分在弹性盒子**两侧**
space-evenly	弹性盒子沿主轴均匀排列，弹性盒子与容器之间间距相等

```
<style>
        .box {
            height: 300px;
            border: 1px solid black;

            display: flex;

            /*justify-content: flex-start*/;

            /*justify-content: flex-end;*/

            /*弹性盒子居中排列*/
            /*justify-content: center;*/

            /*父级剩余的尺寸分配成间距*/

            /*弹性盒子之间的距离相等*/
            /* justify-content: space-between;*/

            /*间距出现在弹性盒子两侧
            视觉效果：弹性盒子之间的间距是两端间距的2倍*/
            /* justify-content: space-around;*/

            /*各个间距都相等*/
            justify-content: space-evenly;

        }

        .box div {
            width: 200px;
            height: 100px;
            background-color: pink;
        }
    </style>
<body>
    <div class="box">
        <div>1</div>
        <div>2</div>
        <div>3</div>
    </div>
    </body>
```

3.侧轴对齐方式

- `align-items`： 当前弹性容器内**所有**弹性盒子的侧轴对齐方式（给**弹性容器**设置）

- `align-self`： 单独控制**某个弹性盒子**的侧轴对齐方式（给**弹性盒子**设置）

属性值	效果
stretch	弹性盒子沿着侧轴线被**拉伸至铺满容器**（弹性盒子没有设置侧轴方向尺寸则默认拉伸）
center	弹性盒子沿侧轴**居中**排列
flex-start	弹性盒子从**起点**开始依次排列
flex-end	弹性盒子从**终点**开始依次排列

```
<style>
        .box {
            height: 300px;
            border: 1px solid black;

            display: flex;

            /*弹性盒子在侧轴方向没有尺寸才能拉伸*/
            /*align-items: stretch;*/

            /*侧轴居中*/
            /*align-items: center;*/

            /* 从起点开始排列 */
            /* align-items: flex-start; */

            /* 从终点开始排列 */
            align-items: flex-end;


        }

        /* 第二个div侧轴居中对齐 */
        .box div:nth-child(2) {
            align-self: center;
        }

        .box div {
            width: 200px;
            height: 100px;
            background-color: pink;
        }
    </style>
<body>
    <div class="box">
        <div>1</div>
        <div>2</div>
        <div>3</div>
    </div>
</body>
```

4.修改主轴方向

主轴默认在水平方向，侧轴默认在垂直方向

属性名：**flex-direction**

属性值	效果
row	水平方向，从左向右（默认）
column	**垂直方向，从上向下**
row-reverse	水平方向，从右向左
column-reverse	垂直方向，从下向上

```
    <style>
        .box {
            height: 120px;
            width: 150px;
            background-color: red;

            display: flex;

            /* 修改主轴的方向 垂直方向：侧轴自动变为水平方向 */
            flex-direction: column;

            /* 主轴在垂直方向，视觉效果：垂直居中 */
            justify-content: center;

            /* 侧轴在水平，视觉效果：水平居中 */
            align-items: center;
        }

        img {
            width: 32px;
            height: 32px;
        }
    </style>
<body>
    <div class="box">
        <img src="./111.png">
        <span>媒体</span>
    </div>
</body>
```

5.弹性伸缩比

作用：控制弹性盒子的主轴方向的尺寸。
属性名：**flex**
属性值：整数数字，表示占用**父级剩余尺寸的份数**。

```
    <style>
        /* 默认情况下，主轴方向尺寸靠内容撑开，侧轴默认拉伸 */
        .box {
            height: 300px;
            border: 1px solid black;

            display: flex;

            flex-direction: column;

        }

        .box div {
            /* height: 100px; */
            background-color: pink;
        }

        .box div:nth-child(1) {
            width: 200px;
        }

        .box div:nth-child(2) {
            flex: 1;
        }

        .box div:nth-child(3) {
            flex: 2;
        }
    </style>
<body>
    <div class="box">
        <div>1</div>
        <div>2</div>
        <div>3</div>
    </div>
</body>
```

6.弹性盒子换行

弹性盒子可以自动挤压或拉伸，默认情况下，所有弹性盒子都在一行显示。

属性名：**flex-wrap**

属性值

- `wrap`：换行
- `nowrap`：不换行（默认）

```
  <style>
        /* 默认情况下，主轴方向尺寸靠内容撑开，侧轴默认拉伸 */
        .box {
            height: 300px;
            border: 1px solid black;

            display: flex;

            /* 不换行 */
            /*flex-wrap: nowrap; */
            flex-wrap: wrap;
            justify-content: space-between;
        }

        .box div {
            width: 200px;
            height: 100px;
            background-color: pink;
        }
    </style>

<body>
    <div class="box">
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>1</div>
        <div>2</div>
        <div>3</div>
    </div>
</body>
```

7.行内对齐方式

属性名：**align-content**

属性值	效果
flex-start	默认值，弹性盒子从**起点**开始依次排列
flex-end	弹性盒子从**终点**开始依次排列
center	弹性盒子沿主轴**居中**排列
space-between	弹性盒子沿主轴均匀排列，空白间距均分在弹性盒子**之间**
space-around	弹性盒子沿主轴均匀排列，空白间距均分在弹性盒子**两侧**
space-evenly	弹性盒子沿主轴均匀排列，弹性盒子与容器之间间距相等

> 注意：该属性对**单行**弹性盒子模型**无效**。

```
<style>
        /* 默认情况下，主轴方向尺寸靠内容撑开，侧轴默认拉伸 */
        .box {
            height: 300px;
            border: 1px solid black;

            display: flex;

            /* 不换行 */
            /*flex-wrap: nowrap; */
            flex-wrap: wrap;
            justify-content: space-between;

            /* 调整行对齐方式 ：对单行弹性盒子不生效*/
            /* align-content: flex-start; */
            /* align-content: flex-end; */
            /* align-content: center; */

            /* align-content: space-between; */
            /* align-content: space-around; */
            align-content: space-evenly;
        }

        .box div {
            width: 200px;
            height: 100px;
            background-color: pink;
        }
    </style>

<body>
    <div class="box">
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>4</div>
        <div>5</div>
        <div>6</div>
        <div>7</div>
        <div>8</div>
        <div>9</div>
        <div>10</div>
    </div>
</body>
```

五、背景属性

1.背景色

`background-color`:背景色

2.背景图

网页中，使用背景图实现装饰性的图片效果
属性名：`background-image`（bgi）
属性值：`url(背景图URL)`

```
<style>
/*盒子宽：400，高：400*/
    div{
        width:400;
        height:400;
    /*背景图默认是平铺（复制）效果*/
        background-image:url(./CSS笔记/image.png)
    }
</style>

<body>
    <div >div1</div>
    <div >div2</div>
</body>
```

A.背景图平铺方式

属性名：`background-repeat`（bgr）
属性值：
1. `no-repeat`：不平铺
2. `repeat`：平铺（默认效果）
3. `repeat-x`：水平方向平铺
4. `repeat-y`：垂直方向平铺

```
<style>
/*盒子宽：400，高：400*/
    div{
        width:400px;
        height:400px;
        background-color:pink;
    /*背景图默认是平铺（复制）效果*/
        background-image:url(./CSS笔记/image.png)
        background-repeat:no-repeat;
    }
</style>

<body>
    <div >div1</div>
</body>
```

背景图位置

属性名：`background-position`（bgp）
属性值：水平方向位置 垂直方向位置
a.关键字
1. `left`：左侧
2. `right`：右侧
3. `center`:居中
4. `top`：顶部
5. `bottom`：底部

b.坐标（`数字+px`，正负都可以
水平：正数向右，负数向左
垂直：正数向下，负数向上

提示：
1.关键字取值方式写法，可以颠倒取值顺序
2.可以只写一个关键字，另一个方向默认为居中；数字只写一个值表示水平方向，垂直方向为居中

```
<style>
/*盒子宽：400，高：400*/
    div{
        width:400px;
        height:400px;
        background-color:pink;
    /*背景图默认是平铺（复制）效果*/
        background-image:url(./CSS笔记/image.png)
        background-repeat:no-repeat;

        /*左上角*/
       /* background-position:0 0;*/
       /* background-position：left top;*/

       /*右下角*/
       /*background-position:right bottom;*/

       /*水平：正数向右，负数向左*/
       /*background-position:50px 0;*/
       /*background-position:-50px 0;*/

       /*垂直：正数向下，负数向上*/
       /*background-position:0 100px;*/
       /*background-position:0 -100px;*/

        /*关键字和数字可以混用*/
       /*background-position:50px center;*/

        /*关键字可以颠倒*/
        /*background-position:bottom left;*/

        /*关键字可以只写一个，另一个方向居中*/
        /*background-position:bottom;*/

        /*只写一个数字表示水平方向，垂直方向居中*/
        /*background-position:50px;*/

    }
</style>

<body>
    <div >div1</div>
</body>
```

B.背景图缩放

作用：设置背景图大小
属性名：`background-size`（bgz）
常用属性值：
a.关键字：
1.`cover`：等比例缩放背景图以完全覆盖背景区，可能背景图片部分看不见
2.`contain`：等比例缩放背景图以完全装入背景区，可能背景区部分空白
b.百分比：根据盒子尺寸计算图片大小
c.`数字 + 单位（例如：px）`

```
<style>
/*盒子宽：400，高：400*/
    div{
        width:400px;
        height:400px;
        background-color:pink;
    /*背景图默认是平铺（复制）效果*/
        background-image:url(./CSS笔记/image.png)
        background-repeat:no-repeat;

        /*contain:如果图的宽度跟盒子尺寸相等停止缩放，可能导致盒子有露白*/
        /*background-size:contain;*/

        /*cover：图片完全覆盖盒子，可能导致图片显示不全*/
        /*background-size:cover;*/


        /*background-size:100%;*/

    }
</style>

<body>
    <div >div1</div>
</body>
```

C.背景图固定

作用：背景不会随着元素的内容滚动
属性名：`background-attachment`(bga)
属性值：`fixed`

```
<style>
    body{
    /*背景图默认是平铺（复制）效果*/
        background-image:url(./CSS笔记/image.png)
        background-repeat:no-repeat;
        background-position:center top;

        background-attachment: fixed;
    }
</style>

<body>
   <p>这是p标签。动作：图二按照图一的动作生成一模一样的动作，左手放在下面，右手指到天空的旁边，保持人
        背景：帮我生成图片：将背景换为灰色墙壁 带有横向分格线 分格线相距稍微远一点 地面换为白色边缘线
        ...
</body>
```

D.背景复合属性

属性名：`background`（bg）
属性值：背景色 背景图 背景图平铺方式 背景图位置/背景图缩放 背景图固定（空格隔开各个属性值，不区分顺序）

```
<style>
    body{
        background: pink url(./CSS笔记/image.png) no-repeat center bottom/cover;
    }
</style>

<body>
   <p>这是p标签。动作：图二按照图一的动作生成一模一样的动作，左手放在下面，右手指到天空的旁边，保持人
        背景：帮我生成图片：将背景换为灰色墙壁 带有横向分格线 分格线相距稍微远一点 地面换为白色边缘线
        ...
</body>
```

六、显示模式

显示模式：标签（元素）的显示方式。
作用：布局网页的时候，根据标签的显示模式选择合适的标签摆放内容。

1.块级元素

a.独占一行
b.宽度默认是父级的100%
c.添加宽高属性生效

```
<style>
/*块级：独占一行；宽度默认是父级100%；加宽高是生效的*/
    div{
        width:100px;
        height:100px;
    }
</style>

<body>
    <div>标签1</div>
    <div>这是div标签2</div>
</body>
```

2.行内元素

a.一行可以显示多个
b.设置宽高属性不生效
c.宽高尺寸由内容撑开

```
<style>
/*行内：一行共存多个；尺寸由内容撑开；加宽高不生效,背景色生效*/
    span{
        width:20px;
        height:20px;
        background-color=red;
    }
</style>

<body>
    <span>span11111111111</span>
    <span>span2</span>
</body>
```

3.行内块元素

a.一行可以显示多个
b.设置宽高属性生效
c.宽高尺寸由内容撑开

```
<style>
/*行内块：一行共存多个；默认尺寸由内容撑开；加宽高生效*/
    img{
        width:20px;
        height:20px;
        background-color=red;
    }
</style>

<body>
    <img src="./image.png">
    <img src="./image.png">
</body>
```

4.转换显示模式

属性名：`display`
属性值：
1.`block`：块级
2.`inline-block`：行内块
3.`inline`：行内
(1和2工作常用)

```
<style>
    div{
        width:100px;
        height:100px;

        display:inline-block;
    }

    span{
        width:20px;
        height:20px;
        background-color=red;

        display: block;

    }

    img{
        width:20px;
        height:20px;
        background-color=red;

        display:block;
    }
</style>

<body>
    /*块级元素*/
    <div>标签1</div>
    <div>这是div标签2</div>

    /*行内元素*/
    <span>span11111111111</span>
    <span>span2</span>

    /*行内块元素*/
    <img src="./image.png">
    <img src="./image.png">
</body>
```

七、CSS高级

> 目标：掌握定位的作用及特点；掌握 CSS 高级技巧

01-定位

作用：灵活的改变盒子在网页中的位置

实现：
1.定位模式：`position`
2.边偏移：设置盒子的位置

- left
- right
- top
- bottom

相对定位

position: relative
特点：

- 不脱标，占用自己原来位置
- 显示模式特点保持不变
- 设置边偏移则相对自己原来位置移动

```
div {
  position: relative;
  top: 100px;
  left: 200px;
}
```

绝对定位

position: absolute
使用场景：子级绝对定位，父级相对定位（**子绝父相**）



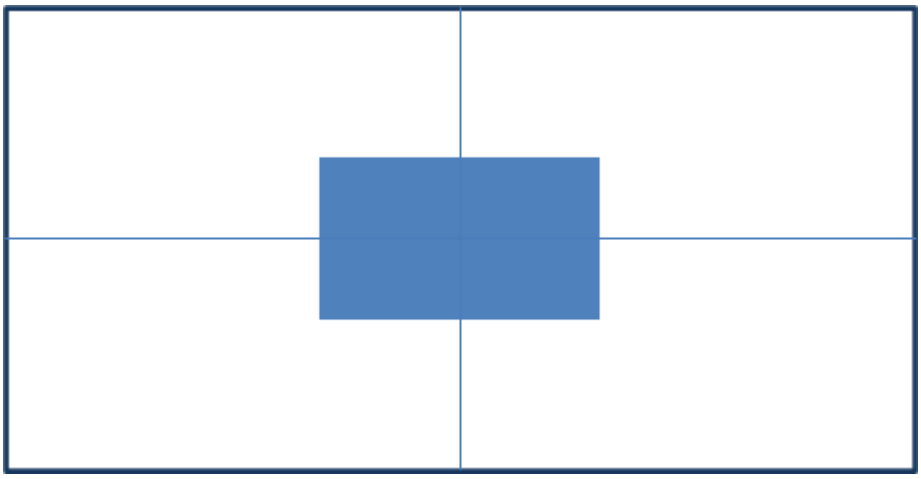
特点：

- 脱标，不占位
- 显示模式具备行内块特点(加宽高生效)
- 设置边偏移则相对最近的已经定位的祖先元素改变位置
- 如果祖先元素都未定位，则相对浏览器可视区改变位置

```
.father {
  position: relative;
}

.father span {
  position: absolute;
  top: 0;
  right: 0;
}
```

定位居中



实现步骤：

1. 绝对定位**position: absolute**
2. 水平（`left`或`right`）、垂直（`top`或`bottom`）边偏移为 50%
3. 子级向左、上移动自身尺寸的一半

- 左、上的外边距为 –尺寸的一半
- transform: translate(-50%, -50%)

```css
img {
  position: absolute;
  left: 50%;
  top: 50%;

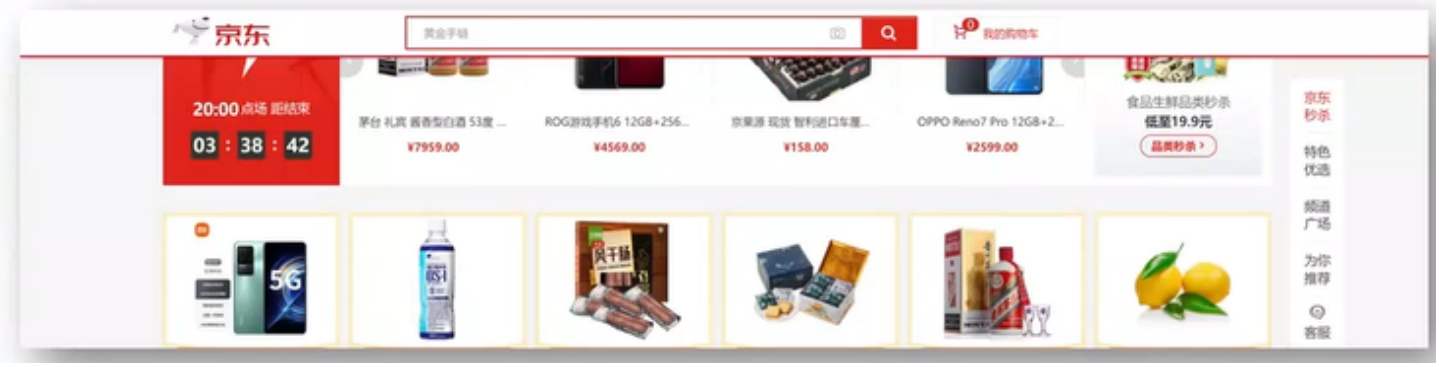
  /* margin-left: -265px;
  margin-top: -127px; */

  /* 方便： 50% 就是自己宽高的一半 */
  transform: translate(-50%, -50%);
}
```

固定定位

position: fixed

场景：元素的位置在网页滚动时不会改变



特点：

- 脱标，不占位
- 显示模式具备行内块特点（加宽高生效）
- 设置边偏移相对浏览器窗口改变位置

```css
div {
  position: fixed;
  top: 0;
  right: 0;

  width: 500px;
}
```

堆叠层级z-index



默认效果：按照标签书写顺序，后来者居上

作用：设置定位元素的层级顺序，改变定位元素的显示顺序

属性名：**z-index**

属性值：**整数数字**（默认值为0，取值越大，层级越高）

```
.box1 {
  background-color: pink;
  /* 取值是整数，默认是0，取值越大显示顺序越靠上 */
  z-index: 1;
}

.box2 {
  background-color: skyblue;
  left: 100px;
  top: 100px;

  z-index: 2;
}
```

总结：

定位-总结

定位模式	属性值	是否脱标	显示模式	参照物
相对定位	relative	否	保持标签原有显示模式	自己原来位置
绝对定位	absolute	是	行内块特点	1. 已经定位的祖先元素 2. 浏览器可视区
固定定位	fixed	是	行内块特点	浏览器窗口

02-高级技巧

CSS精灵

CSS 精灵，也叫 **CSS Sprites**，是一种网页**图片应用处理方式**。把网页中**一些背景图片**整合到**一张图片**文件中，再**background-position** 精确的定位出背景图片的位置。



优点：减少服务器被请求次数，减轻服务器的压力，提高页面加载速度



实现步骤：

1. 创建盒子，**盒子尺寸**与**小图**尺寸**相同**
2. 设置盒子**背景图**为精灵图
3. 添加 **background-position** 属性，改变**背景图位置**
   3.1 使用 PxCook 测量小图片**左上角坐标**

3.2 取**负数**坐标为 `background-position` 属性值（向左上移动图片位置）

案例-京东服务



HTML结构

```
<div class="service">
  <ul>
    <li>
      <h5></h5>
      <p>品类齐全，轻松购物</p>
    </li>
    <li>
      <h5></h5>
      <p>多仓直发，极速配送</p>
    </li>
    <li>
      <h5></h5>
      <p>正品行货，精致服务</p>
    </li>
    <li>
      <h5></h5>
      <p>天天低价，畅选无忧</p>
    </li>
  </ul>
</div>
```

CSS样式

```
<style>
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
  }

  li {
    list-style: none;
  }

  .service {
    margin: 100px auto;
    width: 1190px;
    height: 42px;
    /* background-color: pink; */
  }

  .service ul {
    display: flex;
  }

  .service li {
    display: flex;
    padding-left: 40px;
    width: 297px;
    height: 42px;
    /* background-color: skyblue; */
  }

  .service li h5 {
    margin-right: 10px;
    width: 36px;
    height: 42px;
    /* background-color: pink; */
    background: url(./images/sprite.png) 0 -192px;
  }

  .service li:nth-child(2) h5 {
    background-position: -41px -192px;
  }

  .service li:nth-child(3) h5 {
    background-position: -82px -192px;
```

```
  }

  .service li:nth-child(4) h5 {
    background-position: -123px -192px;
  }

  .service li p {
    font-size: 18px;
    color: #444;
    font-weight: 700;
    line-height: 42px;
  }
</style>
```

字体图标



字体图标：**展示的是图标，本质是字体**

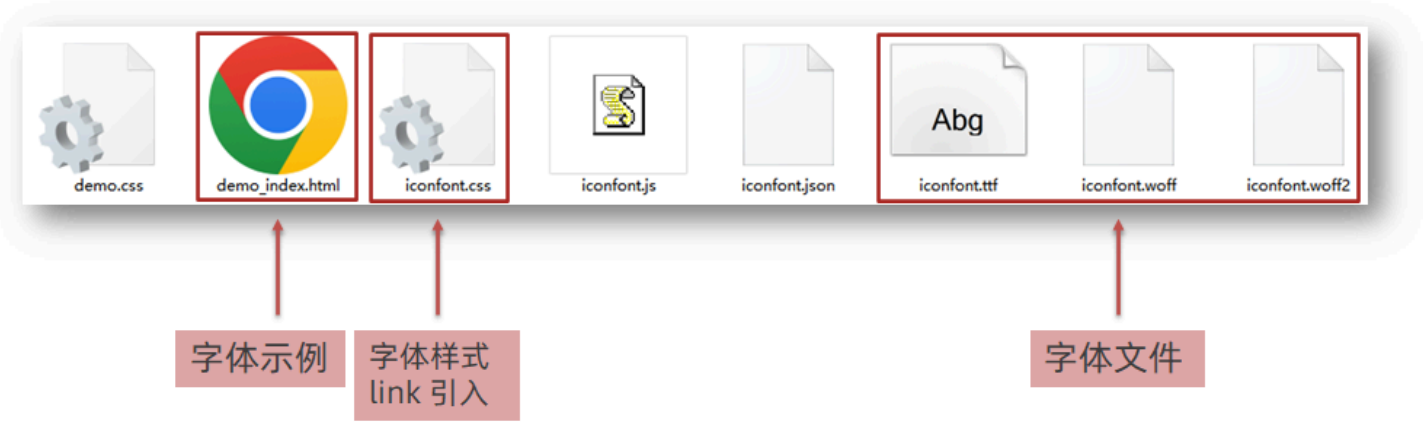
作用：在网页中添加**简单的、颜色单一**的小图标

优点

- **灵活性**：灵活地修改样式，例如：尺寸、颜色等
- **轻量级**：体积小、渲染快、降低服务器请求次数
- **兼容性**：几乎兼容所有主流浏览器
- **使用方便**：先下载再使用

下载字体

iconfont 图标库：https://www.iconfont.cn/
登录 → 素材库 → 官方图标库 → 进入图标库 → 选图标，加入购物车 → 购物车，添加至项目，确定 → 下载至本地



使用字体

1. 引入字体样式表（iconfont.css）

```css
@font-face {
  font-family: "iconfont"; /* 字体名称 */
  src: url('iconfont.woff2?t=1675303337325') format('woff2'),
       url('iconfont.woff?t=1675303337325') format('woff'),
       url('iconfont.ttf?t=1675303337325') format('truetype');
}
```

2. 标签使用字体图标类名
   - iconfont：字体图标基本样式（字体名，字体大小等等）
   - icon-xxx：图标对应的类名

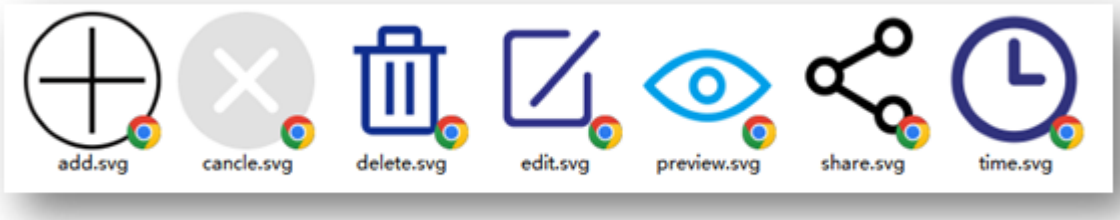
```html
<span class="iconfont icon-xxx"></span>
```

```html
<link rel="stylesheet" href="../CSS实验/iconfont/iconfont.css">
    <title>使用字体图标</title>
    <style>
        /* 如果要调整字体大小，注意 选择器的优先级 要高于 iconfont 类 */
        .iconfont {
            font-size: 200px;
            color: aqua;
        }
    </style>
</head>

<body>
    <span class="iconfont icon-customer-service"></span>
</body>
```

上传矢量图

作用：项目特有的图标上传到 iconfont 图标库，生成字体



上传步骤：上传 → 上传图标 → 选择 svg 矢量图，打开 → 提交 → 系统审核

03-CSS修饰属性

垂直对齐方式



属性名：`vertical-align`

属性值	效果
baseline	基线对齐（默认）
top	顶部对齐
middle	**居中对齐**
bottom	底部对齐

```
  <style>
        div {
            border: 1px solid #000;
        }

        img {

            /* vertical-align: middle; */

            /* 顶对齐：最高内容的 顶部 */
            /* vertical-align: top; */

            /* 底对齐：最高内容的 底部 */
            /* vertical-align: bottom; */

            /* 因为浏览器把行内块、行内标签当作文字处理，默认按基线对齐 */
            /* 效果：图片img的底下有空白，转块级不按基线对齐，空白就消失了 */
            display: block;
        }
    </style>

<body>
    <div>
        <img src="./5.jpg" alt="">
        <!-- why o no!! -->
    </div>
</body>
```

过渡

作用：可以为一个元素在不同状态之间切换的时候添加**过渡效果**

属性名：**transition（复合属性）**

属性值：**过渡的属性 花费时间 (s)**(中间用空格隔开)

提示：

- 过渡的属性可以是具体的 CSS 属性
- 也可以为 `all` （两个状态属性值不同的所有属性，都产生过渡效果）
- transition 设置给元素本身

```
img {
  width: 200px;
  height: 200px;
  transition: all 1s;
}

img:hover {
  width: 500px;
  height: 500px;
}
```

透明度opacity

作用：设置**整个元素的透明度**（包含背景和内容）

属性名：`opacity`

属性值：0 – 1

- 0：完全透明（元素不可见）
- 1：不透明
- 0-1之间小数：半透明

```
<style>
        div {
            width: 500px;
            height: 500px;
            background-color: orange;

            /* opacity: 0; */
            /* opacity: 1; */
            opacity: 0.3;
        }
    </style>

<body>
    <div>
        <img src="./5.jpg" alt="">
    </div>
</body>
```

光标类型cursor

作用：鼠标悬停在元素上时指针显示样式

属性名：`cursor`

属性值	效果
default	默认值，通常是箭头
pointer	小手效果，提示用户可以点击
text	工字型，提示用户可以选择文字
move	十字光标，提示用户可以移动

```
<style>
        div {
            width: 500px;
            height: 500px;
            background-color: orange;

            cursor: pointer;
            cursor: text;
            cursor: move;
        }
    </style>

<body>
    <div></div>
</body>
```

04-综合案例-轮播图



图片效果

HTML结构

```
<div class="banner">
    <!-- 图: ul > li -->
    <ul>
      <li><a href="#"><img src="./images/banner1.jpg" alt=""></a></li>
      <li><a href="#"><img src="./images/banner2.jpg" alt=""></a></li>
      <li><a href="#"><img src="./images/banner3.jpg" alt=""></a></li>
    </ul>
</div>
```

CSS样式

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

li {
  list-style: none;
}

.banner {
  position: relative;
  margin: 100px auto;
  width: 564px;
  height: 315px;
  /* background-color: pink; */
  overflow: hidden;
}

/* 图片 */
.banner img {
  width: 564px;
  border-radius: 12px;
  vertical-align: middle;
}

.banner ul {
  display: flex;
}
```

箭头

HTML结构

```
<!-- 箭头 -->
<!-- 上一张 prev -->
<a href="#" class="prev">
  <i class="iconfont icon-zuoce"></i>
</a>
<!-- 下一张 next -->
<a href="#" class="next">
  <i class="iconfont icon-youce"></i>
</a>
```

CSS样式

```
/* 箭头 */
.banner .prev,
.banner .next {
  /* 隐藏 */
  display: none;
  position: absolute;
  top: 50%;
  transform: translateY(-50%);

  width: 20px;
  height: 30px;
  background-color: rgba(0,0,0, 0.3);

  text-decoration: none;
  color: #fff;
  line-height: 30px;
}

/* 鼠标滑到banner区域，箭头要显示 display:block */
.banner:hover .prev,
.banner:hover .next {
  display: block;
}

.banner .prev {
  left: 0;
  border-radius: 0 15px 15px 0;
}

.banner .next {
  right: 0;
  border-radius: 15px 0 0 15px;
  text-align: center;
}
```

圆点

HTML结构

```
<!-- 圆点 -->
<ol>
  <li></li>
  <li class="active"></li>
  <li></li>
</ol>
```

CSS样式

```
/* 圆点 */
.banner ol {
  position: absolute;
  bottom: 20px;
  left: 50%;
  transform: translateX(-50%);
  height: 13px;
  background-color: rgba(255,255,255,0.3);

  display: flex;

  border-radius: 10px;
}

.banner ol li {
  margin: 3px;
  width: 8px;
  height: 8px;
  background-color: #fff;
  border-radius: 50%;
  cursor: pointer;
}

/* 橙色的li */
.banner ol .active {
  background-color: #ff5000;
}
```

八、移动WEB

移动 Web 第一天

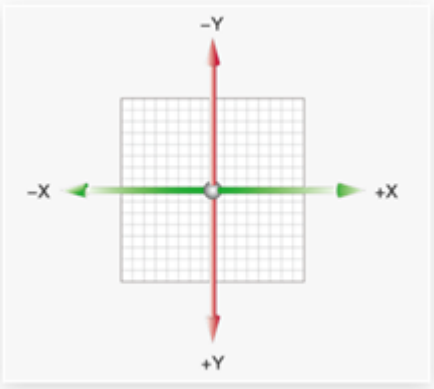
> 目标：使用位移、缩放、旋转、渐变效果丰富网页元素的呈现方式。

01-平面转换

简介

作用：为元素添加动态效果，一般与过渡配合使用

概念：改变盒子在平面内的形态（位移、旋转、缩放、倾斜）



平面转换也叫 2D 转换，属性是 **transform**

```
 <style>
        div {
            margin: 100px 0;
            width: 100px;
            height: 100px;
            background-color: orange;

            transition: all 1s;
        }
        /*添加鼠标悬停效果 */
        div:hover {
            transform: translate(600px) rotate(360deg) scale(2) skew(180deg);
        }
    </style>

<body>
    <div></div>
</body>
```

平移

```
transform: translate(X轴移动距离, Y轴移动距离);
```

- 取值
  - 像素单位数值
  - 百分比（参照**盒子自身尺寸计算结果）
  - 正负均可
- 技巧
  - translate() 只写一个值，表示沿着 X 轴移动
  - 单独设置 X 或 Y 轴移动距离：`translateX()` 或 `translateY()`

```
<style>
        .father {
            margin: 100px auto;
            width: 500px;
            height: 300px;
            border: 1px solid #000;


        }

        .son {
            width: 200px;
            height: 100px;
            background-color: pink;
            transition: all 0.5s;
        }

        /* 鼠标移入到父盒子 .son 改变位置 */
        .father:hover .son {
            transform: translate(200px, 100px);

            /* 取值是百分比：参照盒子自身尺寸计算结果 */
            transform: translate(50%, 100%);
            transform: translate(-50%, 100%);

            /* 只写一个数表示水平方向 */
            transform: translate(100px);

            transform: translateX(50px);
            transform: translateY(100px);
        }
    </style>

<body>
    <div class="father">
        <div class="son"></div>
    </div>
</body>
```

定位居中

- 方法一：margin

```css
position: absolute;
left: 50%;
top: 50%;
/* margin */
margin-left: -100px;
margin-top: -50px;

width: 200px;
height: 100px;
```

- 方法二：平移 → 百分比参照盒子自身尺寸计算结果

```css
position: absolute;
left: 50%;
top: 50%;
transform: translate(-50%, -50%);
```

案例-双开门



- HTML 结构

```html
<div class="father">
    <div class="left"></div>
    <div class="right"></div>
</div>
```

- CSS 样式

```
* {
    margin: 0;
    padding: 0;
}

/* 1. 布局：父子结构，父级是大图，子级是左右小图 */
.father {
    display: flex;
    margin: 0 auto;
    width: 1366px;
    height: 600px;
    background-image: url(./images/bg.jpg);

    overflow: hidden;
}

.father .left,
.father .right {
    width: 50%;
    height: 600px;
    background-image: url(./images/fm.jpg);

    transition: all .5s;
}

.father .right {
    /* right 表示的取到精灵图右面的图片 */
    background-position: right 0;
}

/* 2. 鼠标悬停的效果：左右移动 */
.father:hover .left {
    transform: translate(-100%);
}

.father:hover .right {
    transform: translateX(100%);
}
```

旋转

```
transform: rotate(旋转角度);
```

- 取值：角度单位是 `deg`

- 技巧
  - 取值正负均可
  - 取值为正，顺时针旋转
  - 取值为负，逆时针旋转

```
<style>
        img {
            width: 200px;
            transition: all 1s;
        }

        img:hover {
            transform: rotate(360deg);
            transform: rotate(-360deg);

        }
    </style>
</head>

<body>
    <img src="./rotate.png" alt="">

</body>
```

转换原点

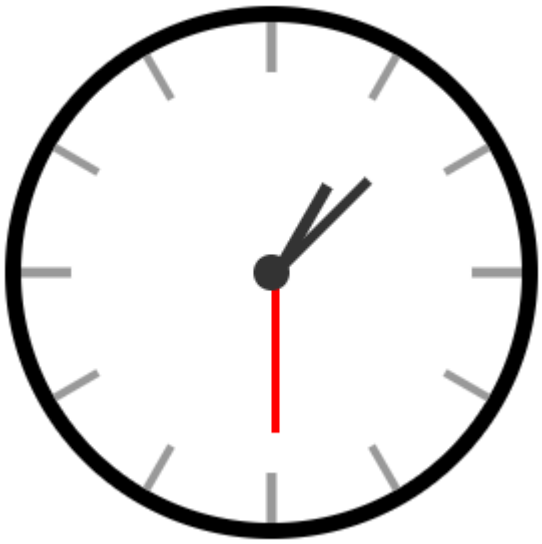
> 默认情况下，转换原点是盒子中心点

```
transform-origin: 水平原点位置 垂直原点位置;
```

取值：

- 方位名词（left、top、right、bottom、center）
- 像素单位数值
- 百分比

案例-时钟



```css
.hour {
  width: 6px;
  height: 50px;
  background-color: #333;
  margin-left: -3px;
  transform: rotate(15deg);
  transform-origin: center bottom;
}

.minute {
  width: 5px;
  height: 65px;
  background-color: #333;
  margin-left: -3px;
  transform: rotate(90deg);
  transform-origin: center bottom;
}

.second {
  width: 4px;
  height: 80px;
  background-color: red;
  margin-left: -2px;
  transform: rotate(240deg);
  transform-origin: center bottom;
}
```

多重转换

多重转换技巧：先平移再旋转

```
transform: translate() rotate();
```

- 多重转换原理：以第一种转换方式坐标轴为准转换形态
  - 旋转会改变网页元素的坐标轴向
  - 先写旋转，则后面的转换效果的轴向以旋转后的轴向为准，会影响转换结果

```
    <title>大风车旋转</title>
    <style>
        .father {
            width: 800px;
            height: 200px;
            border: 1px solid #000;
            transition: all 1s;

            transform-origin: right bottom;
        }

        img {
            width: 200px;
            height: 200px;
            transition: all 1s;
        }


        img:hover {
          /* 先平移再旋转 */
        /* 多重转换：以第一种转换方式坐标轴为准 */
            transform: translate(600px) rotate(360deg);

            /* 层叠性 */
            /* transform: translate(600px); */
            /* transform: rotate(360deg); */


        }
    </style>
</head>

<body>
    <div class="father">
        <div class="child"><img src="./rotate.png" alt=""></div>
    </div>

</body>
```

缩放

```
transform: scale(缩放倍数);
transform: scale(X轴缩放倍数, Y轴缩放倍数);
```

- 技巧
  - 通常，只为 scale() 设置一个值，表示 X 轴和 Y 轴等比例缩放
  - 取值大于1表示放大，取值小于1表示缩小

```
<title>缩放效果</title>
  <style>
    .box {
      width: 300px;
      height: 210px;
      margin: 100px auto;
      background-color: pink;
    }

    .box img {
      width: 100%;
      transition: all 0.5s;
    }

    .box:hover img {
      /* 修改宽高尺寸，从左上角开始缩放 */
      /* width: 500px;
        height: 400px; */

      /* 大于1，表示放大 */
      /* transform: scale(2); */

      /* 小于1，表示缩小 */
      transform: scale(0.5);

      /* 等于1，不变 */
      /* transform: scale(1); */
    }
  </style>
</head>

<body>
  <div class="box">
    <img src="/assets/1681358481436.png" alt="" />
  </div>
</body>
```

案例-播放特效



【和平精英】“初火”音乐概念片：四圣觉醒.....

- CSS 样式

```css
/* 1. 摆放播放按钮：图片区域的中间 */
.box li {
  overflow: hidden;
}

.pic {
  position: relative;
}

.pic::after {
  position: absolute;
  left: 50%;
  top: 50%;
  /* margin-left: -29px;
  margin-top: -29px; */
  /* transform: translate(-50%, -50%); */

  content: '';
  width: 58px;
  height: 58px;
  background-image: url(./images/play.png);
  transform: translate(-50%, -50%) scale(5);
  opacity: 0;

  transition: all .5s;
}
/* 2. hover效果：大按钮，看不见：透明是0 → 小按钮，看得见：透明度1 */
.box li:hover .pic::after {
  transform: translate(-50%, -50%) scale(1);
  opacity: 1;
}
```

```
<title>按钮缩放</title>
<style>
  * {
    margin: 0;
    padding: 0;
  }

  li {
    list-style: none;
  }

  img {
    width: 100%;
  }

  .box {
    width: 249px;
    height: 210px;
    margin: 50px auto;
  }

  .box p {
    color: #3b3b3b;
    padding: 10px 10px 0 10px;
  }

  /* 1. 摆放播放按钮：图片区域的中间 */
  .box li {
    overflow: hidden;
  }

  .pic {
    position: relative;
  }

  .pic::after {
    position: absolute;
    left: 50%;
    top: 50%;
    /* margin-left: -29px;
    margin-top: -29px; */
    /* transform: translate(-50%, -50%); */

    content: '';
    width: 58px;
    height: 58px;
```

```
        background-image: url(./images/play.png);
        transform: translate(-50%, -50%) scale(5);
        opacity: 0;

        transition: all .5s;
      }
      /* 2. hover效果：大按钮，看不见：透明是0 → 小按钮，看得见：透明度1 */
      .box li:hover .pic::after {
        transform: translate(-50%, -50%) scale(1);
        opacity: 1;
      }
    </style>
  </head>
  <body>
    <div class="box">
      <ul>
        <li>
          <div class="pic">
            <img src="./images/party.jpeg" alt="" />
          </div>
          <p>【和平精英】“初火”音乐概念片：四圣觉醒......</p>
        </li>
      </ul>
    </div>
  </body>
```

倾斜

```
transform: skew();
```

取值：角度度数 deg

```
<title>倾斜效果</title>
    <style>
      div {
        margin: 100px auto;
        width: 100px;
        height: 200px;
        background-color: pink;
        transition: all 0.5s;
      }

      div:hover {
        transform: skew(30deg);
        transform: skew(-30deg);
      }
    </style>
  </head>
  <body>
    <div></div>
  </body>
```

02-渐变

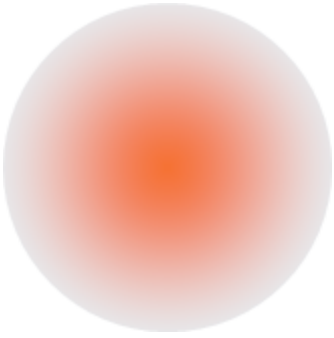
渐变是多个颜色逐渐变化的效果，一般用于设置盒子背景

分类：

- 线性渐变



- 径向渐变



线性渐变

```
background-image: linear-gradient(
  渐变方向,
  颜色1 终点位置,
  颜色2 终点位置,
  ......
);
```

取值：

- 渐变方向：可选
  - to 方位名词
  - 角度度数
- 终点位置：可选
  - 百分比

```
<title>渐变-线性</title>
    <style>
      div {
        width: 200px;
        height: 200px;
        background-color: green;
        background-image: linear-gradient(
          red,
          green
        );
        background-image: linear-gradient(
          to right,
          red,
          green
        );
        background-image: linear-gradient(
          45deg,
          red,
          green
        );
        background-image: linear-gradient(
          red 80%,
          green
        );
      }
    </style>
  </head>
  <body>
    <div></div>
  </body>
```

案例-产品展示



- HTML 结构

```
<div class="box">
  <img src="./images/product.jpeg" alt="" />
  <div class="title">OceanStor Pacific 海量存储斩获2021 Interop金奖</div>
  <div class="mask"></div>
</div>
```

- CSS 样式

```
.mask {
  position: absolute;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  background-image: linear-gradient(
      transparent,
      rgba(0,0,0,0.5)
  );
  opacity: 0;

  transition: all .5s;
}

.box:hover .mask {
  opacity: 1;
}
```

径向渐变

```
background-image: radial-gradient(
  半径 at 圆心位置,
  颜色1 终点位置,
  颜色2 终点位置,
  ......
);
```

取值：

- 半径可以是2条，则为椭圆
- 圆心位置取值：像素单位数值 / 百分比 / 方位名词
- 终点位置：可选
    - 百分比

```
<title>渐变-径向</title>
  <style>
    div {
      width: 100px;
      height: 100px;
      background-color: pink;
      border-radius: 50%;
      background-image: radial-gradient(50px at center center,
          red,
          pink);
      background-image: radial-gradient(50px 20px at center center,
          red,
          pink);
      background-image: radial-gradient(50px at 50px 30px,
          red,
          pink 50%);
    }

    button {
      width: 100px;
      height: 40px;
      background-color: green;
      border: 0;
      border-radius: 5px;
      color: #fff;
      background-image: radial-gradient(30px at 30px 20px,
          rgba(255, 255, 255, 0.2),
          transparent);
    }
  </style>
</head>

<body>
  <div></div>
  <button>按钮</button>
</body>
```

03-综合案例



导航-频道

箭头旋转

```
.x-header-nav .nav-item:hover .icon-down {
  transform: rotate(-180deg);
}
```

频道列表

```
.channel-layer {
  position: absolute;
  top: 60px;
  left: 50%;
  z-index: -2;
  width: 1080px;
  height: 120px;
  padding: 10px;
  margin-left: -540px;
  color: #72727b;
  background-color: #f5f5f5;
  border: 1px solid #e4e4e4;
  border-top: none;
  transition: all 0.5s;
  transform: translateY(-120px);
}

/* TODO 2. 弹窗频道 */
.x-header-nav .nav-item:hover .channel-layer {
  transform: translateY(0);
}
```

渐变按钮

搜索按钮

```
.x-header-search form .btn {
  position: absolute;
  top: 0;
  right: 0;
  width: 60px;
  height: 40px;
  line-height: 40px;
  text-align: center;
  background-color: #f86442;
  border-top-right-radius: 20px;
  border-bottom-right-radius: 20px;
  background-image: linear-gradient(
    to right,
    rgba(255, 255, 255, 0.3),
    #f86442
  );
}
```

登录按钮

```
/* TODO 7. 渐变按钮 */
.card .card-info .login {
  padding: 3px 34px;
  color: #fff;
  background-color: #ff7251;
  border-radius: 30px;
  box-shadow: 0 4px 8px 0 rgb(252 88 50 / 50%);
  background-image: linear-gradient(
    to right,
    rgba(255, 255, 255, 0.2),
    #ff7251
  );
}
```

客户端按钮

```
/* TODO 8. 径向渐变 */
.download .dl .dl-btn {
  width: 68px;
  height: 34px;
  line-height: 34px;
  color: #fff;
  text-align: center;
  border-radius: 4px;
  background-image: radial-gradient(
    50px at 10px 10px,
    rgba(255, 255, 255, 0.5),
    transparent
  );
}
```

轮播图

```
/* TODO 4. 摆放图片 */
.banner .banner-list .banner-item.left {
  z-index: 0;
  transform: translate(-160px) scale(0.8);
  transform-origin: left center;
}

.banner .banner-list .banner-item.right {
  z-index: 0;
  transform: translate(160px) scale(0.8);
  transform-origin: right center;
}
```

猜你喜欢

```
/* TODO 5. 播放按钮和遮罩 */
.album-item .album-item-box::after {
  position: absolute;
  left: 0;
  top: 0;
  content: '';
  width: 100%;
  height: 100%;
  background: rgba(0,0,0,.5) url(../assets/play.png) no-repeat center / 20px;
  opacity: 0;
  transition: all .5s;
}

.album-item .album-item-box:hover::after {
  opacity: 1;
  background-size: 50px;
}

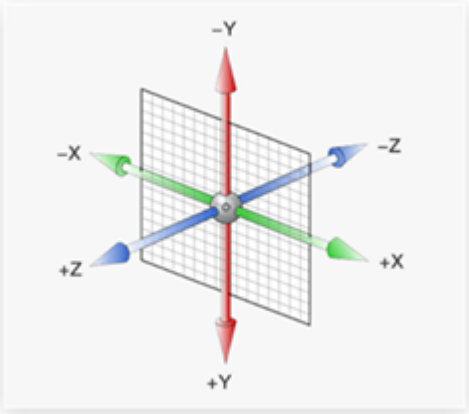

/* TODO 6. 图片缩放 */
.album-item .album-item-box:hover img {
  transform: scale(1.1);
}
```

移动 Web 第二天

04-空间转换

空间转换简介

- 空间：是从坐标轴角度定义的X 、Y 和 Z 三条坐标轴构成了一个立体空间，Z 轴位置与视线方向相同。
- 空间转换也叫 3D转换
- 属性：transform



平移

```
transform: translate3d(x, y, z);
transform: translateX();
transform: translateY();
transform: translateZ();
```

取值：(正负均可)

- 像素单位数值
- 百分比（参照盒子自身尺寸计算结果）

> 取值与平面转换相同
>
> 默认情况下，Z 轴平移没有效果，原因：电脑屏幕默认是平面，无法显示 Z 轴平移效果

```
<title>空间平移</title>
  <style>
    .box {
      width: 200px;
      height: 200px;
      margin: 100px auto;
      background-color: pink;
      transition: all 0.5s;
    }

    .box:hover {
      /* 电脑是平面，默认无法观察 Z 轴平移效果 */
      /* transform: translate3d(100px, 200px, 300px); */

      /* 3d 小括号里面必须逗号隔开三个数 */
      /* transform: translate3d(100px, 200px); */

      transform: translateX(100px);
      transform: translateY(-100%);
      transform: translateZ(300px);
    }
  </style>
</head>

<body>
  <div class="box"></div>
</body>
```

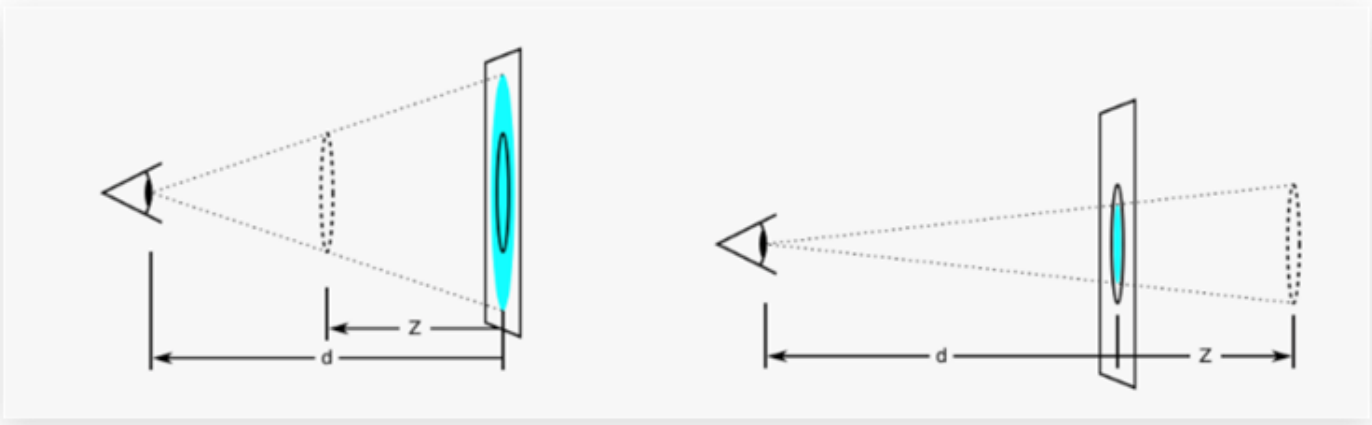
视距

作用：指定了观察者与 Z=0 平面的距离，为元素添加透视效果

透视效果：近大远小、近实远虚

属性：(添加给父级，取值范围 800-1200)

```
perspective: 视距;
```




```
<title>透视效果</title>
  <style>
    /* 视距属性必须添加给 直接父级 */
    .father {
      perspective: 1000px;
      /* perspective: 10000px;
      perspective: 100px; */
    }

    .son {
      width: 200px;
      height: 200px;
      margin: 100px auto;
      background-color: pink;
      transition: all 0.5s;
    }

    .son:hover{
      transform: translateZ(-300px);
      transform: translateZ(300px);
    }
  </style>
</head>

<body>
  <div class="father">
    <div class="son"></div>
  </div>
</body>
```

空间-旋转

- Z 轴：rotateZ()



```
<title>空间旋转-Z轴</title>
    <style>
      .box {
        width: 300px;
        margin: 100px auto;
      }

      img {
        width: 300px;
        transition: all 2s;
      }

      .box img:hover {
        transform: rotateZ(360deg);
      }
    </style>
  </head>
  <body>
    <div class="box">
      <img src="./images/hero.jpeg" alt="" />
    </div>
  </body>
```

- X 轴：rotateX()



```
<title>空间旋转-X轴</title>
    <style>
      .box {
        width: 300px;
        margin: 100px auto;
      }

      img {
        width: 300px;
        transition: all 2s;
      }

      .box {
        /* 透视效果：近大远小，近实远虚 */
        perspective: 1000px;
      }

      .box img:hover {
        transform: rotateX(60deg);
        transform: rotateX(-60deg);
      }
    </style>
  </head>
  <body>
    <div class="box">
      <img src="./images/hero.jpeg" alt="" />
    </div>
  </body>
```

- Y 轴：rotateY()



```
<title>空间旋转-Y轴</title>
    <style>
      .box {
        width: 300px;
        margin: 100px auto;
      }

      img {
        width: 300px;
        transition: all 2s;
      }

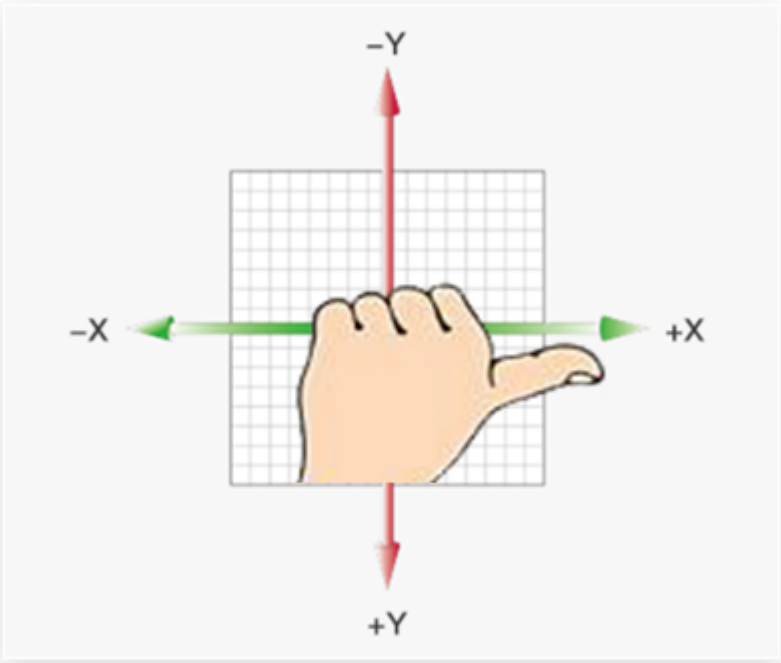
      .box {
        /* 透视效果：近大远小，近实远虚 */
        perspective: 1000px;
      }

      .box img:hover {
        transform: rotateY(60deg);
        transform: rotateY(-60deg);
      }
    </style>
  </head>
  <body>
    <div class="box">
      <img src="./images/hero.jpeg" alt="" />
    </div>
  </body>
```

左手法则

作用：根据旋转方向确定取值正负

使用：左手握住旋转轴, 拇指指向正值方向, 其他四个手指弯曲方向为旋转正值方向



rotate3d-了解

- `rotate3d(x, y, z, 角度度数)` ：用来设置自定义旋转轴的位置及旋转的角度
- x，y，z 取值为0-1之间的数字

立体呈现

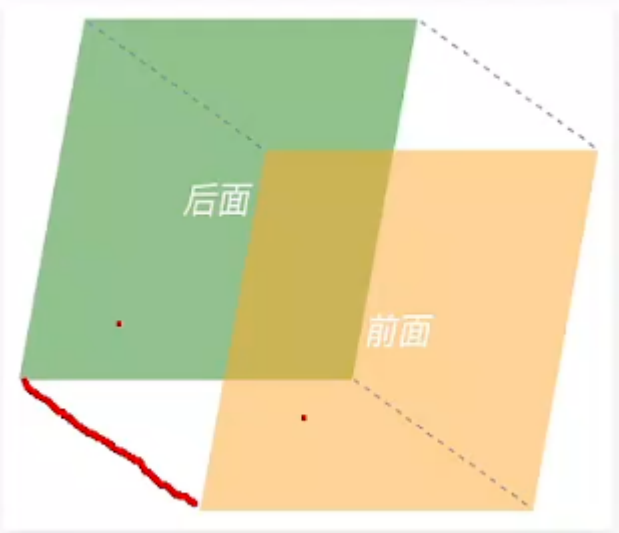
作用：设置元素的子元素是位于 3D 空间中还是平面中

属性名：transform-style

属性值：

- flat：子级处于平面中
- preserve-3d：子级处于 3D 空间。

呈现立体图形步骤
1.父元素添加`transform-style:preserve-3d`;
2.子级定位
3.调整子盒子的位置（位移或旋转）


后面
前面

```
<title>立体呈现</title>
    <style>
      .cube {
        position: relative;
        width: 200px;
        height: 200px;
        margin: 100px auto;
        /* background-color: pink; */
        transition: all 2s;

        transform-style: preserve-3d;

        /* 旋转与案例效果无关，用来看前后移动的效果 */
        /* transform: rotateY(89deg); */
      }

      .cube div {
        position: absolute;
        left: 0;
        top: 0;
        width: 200px;
        height: 200px;
      }

      .front {
        background-color: orange;
        transform: translateZ(100px);
      }

      .back {
        background-color: green;
        transform: translateZ(-100px);
      }

      .cube:hover {
        transform: rotateY(90deg);
      }
    </style>
  </head>
  <body>
    <div class="cube">
      <div class="front">前面</div>
      <div class="back">后面</div>
    </div>
  </body>
```

案例-3d导航



案例步骤：

1. 搭建立方体
    i. 绿色是立方体的前面
    ii. 橙色是立方体的上面
2. 鼠标悬停，立方体旋转





定位　橙色旋转（X）　橙色位移（Z）　绿色位移（Z）

```css
.nav li {
  position: relative;
  width: 100px;
  height: 40px;
  line-height: 40px;
  transition: all 0.5s;

  transform-style: preserve-3d;

  /* 为了看到橙色和绿色的移动过程，给立方体添加旋转 */
  /* transform: rotateX(-20deg) rotateY(30deg); */
}

.nav li a {
  position: absolute;
  left: 0;
  top: 0;
  display: block;
  width: 100%;
  height: 100%;
  text-align: center;
  text-decoration: none;
  color: #fff;
}

/* 立方体每个面都有独立的坐标轴，互不影响 */
.nav li a:first-child {
  background-color: green;
  transform: translateZ(20px);
}

.nav li a:last-child {
  background-color: orange;
  transform: rotateX(90deg) translateZ(20px);
}

.nav li:hover {
  transform: rotateX(-90deg);
}
```

```
<title>3D导航</title>
    <style>
      ul {
        margin: 0;
        padding: 0;
        list-style: none;
      }

      .nav {
        width: 300px;
        height: 40px;
        margin: 50px auto;
      }

      .nav ul {
        display: flex;
      }

      .nav li {
        position: relative;
        width: 100px;
        height: 40px;
        line-height: 40px;
        transition: all 0.5s;

        transform-style: preserve-3d;

        /* 为了看到橙色和绿色的移动过程，给立方体添加旋转 */
        /* transform: rotateX(-20deg) rotateY(30deg); */
      }

      .nav li a {
        position: absolute;
        left: 0;
        top: 0;
        display: block;
        width: 100%;
        height: 100%;
        text-align: center;
        text-decoration: none;
        color: #fff;
      }

      /* 立方体每个面都有独立的坐标轴，互不影响 */
      .nav li a:first-child {
        background-color: green;
```

```
        transform: translateZ(20px);
      }

      .nav li a:last-child {
        background-color: orange;
        transform: rotateX(90deg) translateZ(20px);
      }

      .nav li:hover {
        transform: rotateX(-90deg);
      }
    </style>
  </head>

  <body>
    <div class="nav">
      <ul>
        <li>
          <a href="#">首页</a>
          <a href="#">Index</a>
        </li>
        <li>
          <a href="#">登录</a>
          <a href="#">Login</a>
        </li>
        <li>
          <a href="#">注册</a>
          <a href="#">Register</a>
        </li>
      </ul>
    </div>
  </body>
```

空间缩放

```
transform: scale3d(x, y, z);
transform: scaleX();
transform: scaleY();
transform: scaleZ();
```

例子：

```
.nav li {
      position: relative;
      width: 100px;
      height: 40px;
      line-height: 40px;
      transition: all 0.5s;

      transform-style: preserve-3d;

      /* 为了看到橙色和绿色的移动过程，给立方体添加旋转 */
      /* transform: rotateX(-20deg) rotateY(30deg); */

      /*transform: scaleX(0.5);
      transform: scaleY(2);
      transform: scaleZ(3);*/

      transform: scale3d(0.5, 2, 3);

    }
```

05-动画

- 过渡：实现两个状态间的变化过程
- 动画：实现多个状态间的变化过程，动画过程可控（重复播放、最终画面、是否暂停）

```
<title>体验动画</title>
    <style>
      div {
        width: 100px;
        height: 100px;
        background-color: pink;

        animation: change 1s infinite alternate;
      }

      @keyframes change {
        0% {
          transform: translate(0);
        }

        100% {
          transform: translate(600px);
        }
      }
    </style>
  </head>
  <body>
    <div></div>
  </body>
```

动画实现步骤

1. 定义动画

```
/* 方式一 */
@keyframes 动画名称 {
  from {} /*当前状态*/
  to {}
}

/* 方式二 */
@keyframes 动画名称 {
  0% {}
  10% {}
  ......
  100% {}
}
```

2. 使用动画

```
animation: 动画名称 动画花费时长;
```

```
<title>动画实现步骤</title>
   <style>
     .box {
       width: 200px;
       height: 100px;
       background-color: pink;

       animation: change 1s;
     }

     /* 动画一：宽度从200变化到800 */
     /* @keyframes change {
       from {
           width: 200px;
       }
       to {
           width: 800px;
       }
     } */


     /* 动画二：从 200*100 变化到 300*300 变化到800*500 */
     /* 百分比：表示的意思是动画时长的百分比 */
     @keyframes change {
       0% {
           width: 200px;
           height: 100px;
       }
       20% {
           width: 300px;
           height: 300px;
       }
       100% {
           width: 800px;
           height: 500px;
       }
     }

   </style>
 </head>
 <body>
   <div class="box"></div>
 </body>
```

animation复合属性

```
animation: 动画名称 动画时长 速度曲线 延迟时间 重复次数 动画方向 执行完毕时状态;
```

提示：

- 动画名称和动画时长必须赋值
- 取值不分先后顺序
- 如果有两个时间值，第一个时间表示动画时长，第二个时间表示延迟时间

```
<title>animation复合属性</title>
  <style>
    .box {
      width: 200px;
      height: 100px;
      background-color: pink;

      /* linear：匀速 */
      animation: change 1s linear;

      /* steps：分步动画，工作中，配合精灵图实现精灵动画 */
      animation: change 1s steps(3);

      /* 如果有两个时间，第一个是动画时长，第二个是延迟时间 */
      animation: change 1s 2s;

      /* 重复次数3，infinite：无限循环 */
      animation: change 1s 3;
      animation: change 1s infinite;

      /* alternate：反向 */
      animation: change 1s infinite alternate;

      /* 动画执行完毕时的状态， forwards：结束状态； backwards：开始状态（默认） */
      animation: change 1s forwards;
      animation: change 1s backwards;
    }

    /* 宽度 从 200 变化到 800 */
    @keyframes change {
      from {
          width: 200px;
      }
      to {
          width: 800px;
      }
    }

  </style>
</head>
<body>
  <div class="box"></div>
</body>
```

animation拆分写法

工作不常用

属性	作用	取值
animation-name	动画名称	
animation-duration	动画时长	
animation-delay	延迟时间	
animation-fill-mode	动画执行完毕时状态	forwards：最后一帧状态 backwards：第一帧状态
animation-timing-function	速度曲线	steps(数字)：逐帧动画
animation-iteration-count	重复次数	infinite为无限循环
animation-direction	动画执行方向	alternate为反向
animation-play-state	暂停动画	paused为暂停，通常配合:hover使用

```
<title>animation拆分写法</title>
    <style>
      .box {
        width: 200px;
        height: 100px;
        background-color: pink;

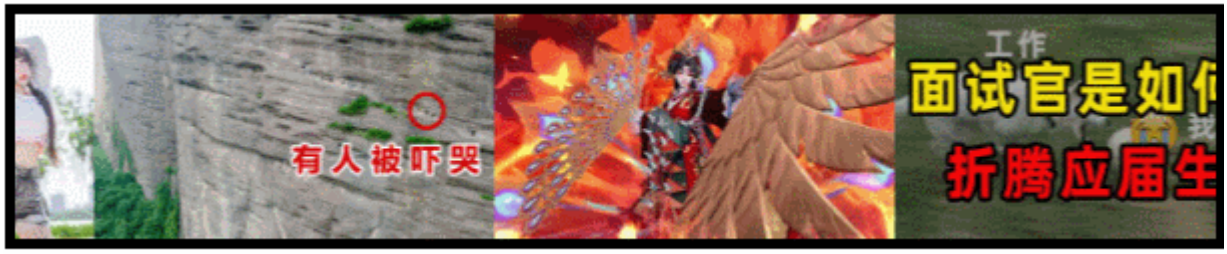
        /* 动画名称 */
        animation-name: change;
        /* 动画时长 */
        animation-duration: 1s;
        /* 播放次数 */
        animation-iteration-count: infinite;

        /* animation-play-state: paused; */
      }

      .box:hover {
        /* 暂停动画 */
        animation-play-state: paused;
      }


      /* 宽度从 200 变化到 800 */
      @keyframes change {
        0% {
            width: 200px;
        }
        100% {
            width: 800px;
        }
      }
    </style>
  </head>
  <body>
    <div class="box"></div>
  </body>
```

案例-走马灯



- HTML 结构

```
<li><img src="./images/1.jpg" alt="" /></li>
<li><img src="./images/2.jpg" alt="" /></li>
<li><img src="./images/3.jpg" alt="" /></li>
<li><img src="./images/4.jpg" alt="" /></li>
<li><img src="./images/5.jpg" alt="" /></li>
<li><img src="./images/6.jpg" alt="" /></li>
<li><img src="./images/7.jpg" alt="" /></li>

<!-- 替身：填补显示区域的露白 -->
<li><img src="./images/1.jpg" alt="" /></li>
<li><img src="./images/2.jpg" alt="" /></li>
<li><img src="./images/3.jpg" alt="" /></li>
```

- CSS 样式

```
.box {
  width: 600px;
  height: 112px;
  border: 5px solid #000;
  margin: 100px auto;

  overflow: hidden;
}

.box ul {
  display: flex;
  animation: move 6s infinite linear;
}

/* 定义位移动画；ul使用动画；鼠标悬停暂停动画 */
@keyframes move {
  0% {
    transform: translate(0);
  }
  100% {
    transform: translate(-1400px);
  }
}

.box:hover ul {
  animation-play-state: paused;
}
```

> 无缝动画原理：复制开头图片到结尾位置（图片累加宽度 = 区域宽度）

精灵动画

- 核心

属性	作用	取值
animation-timing-function	速度曲线	steps(数字)：逐帧动画

1.steps()逐帧动画
2.CSS精灵图

- 制作步骤
  1.准备显示区域
  盒子尺寸与一张精灵小图尺寸相同
  2.定义动画

移动背景图（移动距离 = 精灵图宽度）

3.使用动画

steps(N)，N 与精灵小图个数相同

```
<title>精灵动画</title>
  <style>
    div {
      width: 140px;
      height: 140px;
      border: 1px solid #000;
      background-image: url(./images/bg.png);
      animation: run 1s steps(12) infinite;
    }

    @keyframes run {
      from {
        background-position: 0 0;
      }
      to {
        background-position: -1680px 0;
      }
    }
  </style>
</head>
<body>
  <div></div>
</body>
```

多组动画

```
animation:
  动画一,
  动画二,
  ... ...
;
```

```
<title>多组动画</title>
  <style>
    div {
      width: 140px;
      height: 140px;
      /* border: 1px solid #000; */
      background-image: url(./images/bg.png);
      animation:
        run 1s steps(12) infinite,
        move 3s forwards
      ;
    }

    /* 当动画的开始状态样式 跟 盒子默认样式相同，可以省略动画开始状态的代码 */
    @keyframes run {
      /* from {
        background-position: 0 0;
      } */
      to {
        background-position: -1680px 0;
      }
    }

    @keyframes move {
      /* 0% {
        transform: translate(0);
      } */
      100% {
        transform: translate(800px);
      }
    }
  </style>
</head>
<body>
  <div></div>
</body>
```

06-综合案例-全名出游



背景

```
/* 大背景 */
/* 默认状态HTML和body的高度是0，所以导致cover缩放背景图不成功 */
html {
  height: 100%;
}
body {
  height: 100%;
  background: url(../images/f1_1.jpg) no-repeat center 0 / cover;
  /* background-size: cover; */
}
```

云彩位置和动画

- HTML 结构

```
<!-- 云 -->
<div class="cloud">
  <img src="./images/yun1.png" alt="">
  <img src="./images/yun2.png" alt="">
  <img src="./images/yun3.png" alt="">
</div>
```

- CSS 样式

```css
/* 云 */
.cloud img {
  position: absolute;
  left: 50%;
}

.cloud img:nth-child(1) {
  margin-left: -250px;
  top: 20px;
  animation: cloud 1s infinite alternate linear;
}
.cloud img:nth-child(2) {
  margin-left: 400px;
  top: 100px;
  animation: cloud 1s infinite alternate linear 0.4s;
}
.cloud img:nth-child(3) {
  margin-left: -550px;
  top: 200px;
  animation: cloud 1s infinite alternate linear 0.6s;
}

@keyframes cloud {
  100% {
    transform: translate(20px);
  }
}
```

文字动画

- HTML 结构

```html
<!-- 文字 -->
<div class="text">
  <img src="./images/font1.png" alt="">
</div>
```

- CSS 样式

```css
/* 文字 */
.text img {
  position: absolute;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%);
  animation: text 1s;
}

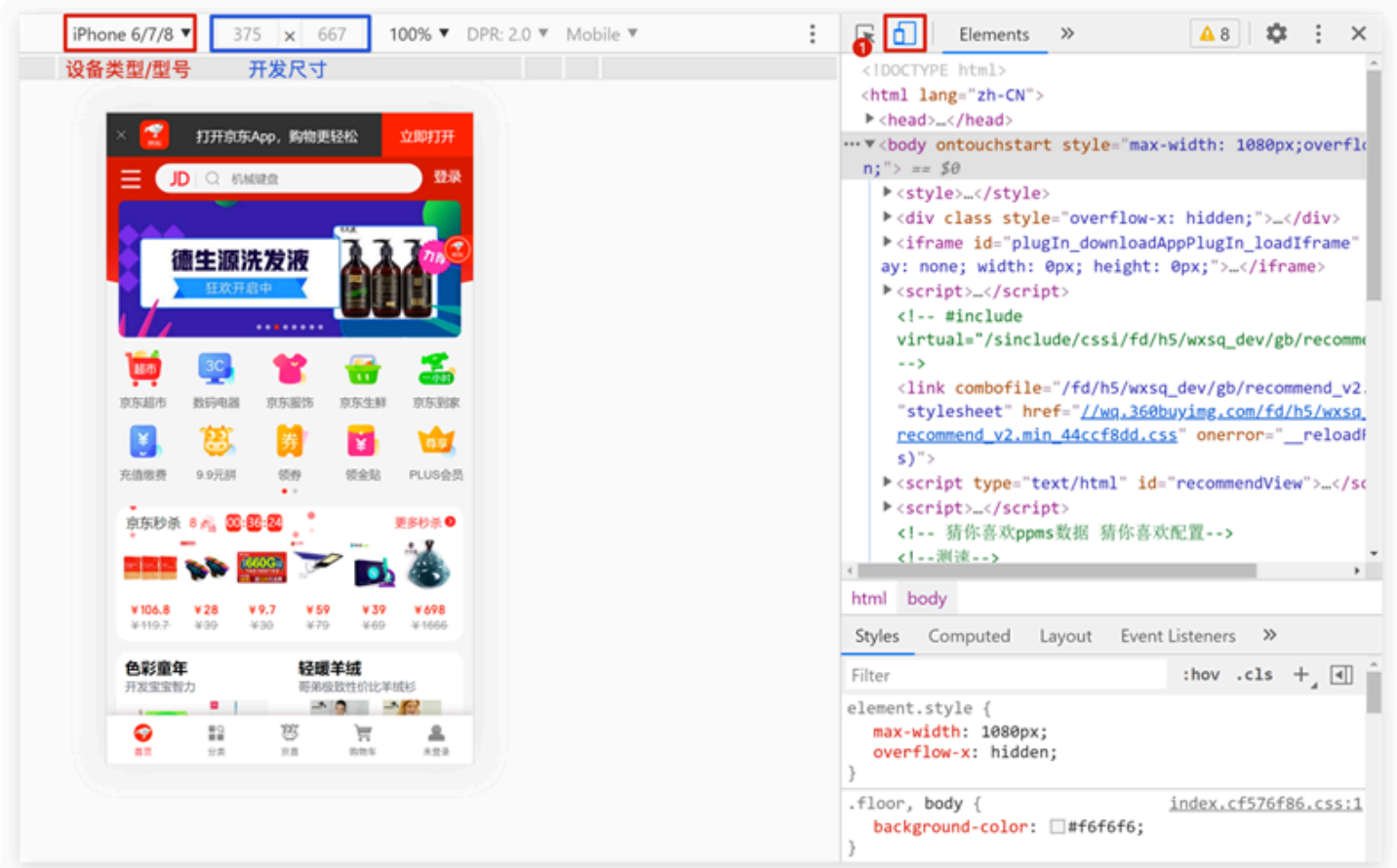
/* 默认 → 小 → 大 → 小 → 默认 */
@keyframes text {
  0% {
    transform: translate(-50%, -50%) scale(1);
  }
  20% {
    transform: translate(-50%, -50%) scale(0.1);
  }
  40% {
    transform: translate(-50%, -50%) scale(1.4);
  }
  70% {
    transform: translate(-50%, -50%) scale(0.8);
  }
  100% {
    transform: translate(-50%, -50%) scale(1);
  }
}
```

移动 Web 第三天

07-移动 Web 基础

谷歌模拟器

模拟移动设备，方便查看页面效果



屏幕分辨率

分类：

- 物理分辨率：硬件分辨率（出厂设置）
- 逻辑分辨率：软件 / 驱动设置

结论：**制作网页参考 逻辑分辨率**

手机型号	物理分辨率	逻辑分辨率	比例关系
iPhone3GS	320 * 480	320 * 480	1 : 1
iPhone4	640 * 960	320 * 480	2 : 1
iPhone5	640 * 1136	320 * 568	2 : 1
iPhone6/7/8	750 * 1334	375 * 667	2 : 1
iPhone6/7/8 Plus	1080 * 1920	414 * 736	2.6 : 1
iPhone11Pro/X/XS	1125 * 2436	375 * 812	3 : 1
iPhone11/XR	828 * 1792	414 * 896	2 : 1
iPhone11 Pro Max/XS Max	1242 * 2688	414 * 896	3 : 1
iPhone12 mini	1080 * 2340	360 * 780	3 : 1
iPhone12/iPhone12 Pro	1170 * 2532	390 * 844	3 : 1
iPhone12 Pro Max	1284 * 2778	428 * 926	3 : 1

视口

作用：显示 HTML 网页的区域，用来约束 HTML 的尺寸

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">

  <!- 视口标签:规定HTML的尺寸，让HTML的宽度 = 逻辑分辨率的宽度/设备的宽度 -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Document</title>
</head>
<body>

</body>
</html>
```

- `width=device-width`：视口宽度 = 设备宽度
- `initial-scale=1.0`：缩放1倍（不缩放）

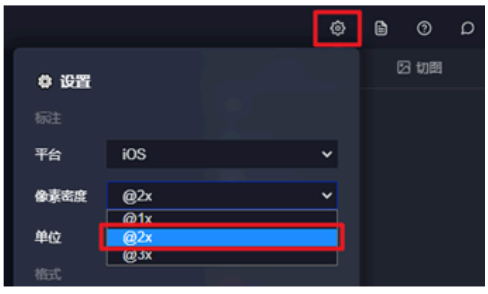
二倍图

概念：设计稿里面每个元素的尺寸的倍数

作用：防止图片在高分辨率屏幕下模糊失真

使用方法：





适配方案

- 宽度适配：宽度自适应（PC端）
  - 百分比布局
  - Flex 布局
- 等比适配：宽高等比缩放（移动端）

- rem
- vw

08-rem

简介

- rem单位，是相对单位
- rem单位是相对于HTML标签的字号计算结果
- 1rem = 1HTML字号大小

```
<title>rem基本使用</title>
    <style>
        * {
            margin: 0;
            padding: 0;
        }

       /* 1.给HTML标签加字号 */
        html {
            font-size: 30px;
        }

        /* 2.使用rem单位书写尺寸 */
        .box {
            width: 5rem;
            height: 3rem;
            background-color: pink;
        }

    </style>
</head>

<body>
    <div class="box"></div>
</body>
```

> 思考手机屏幕大小不同，分辨率不同，如何设置不同的HTML标签字号？
> 答案是：媒体查询

媒体查询

媒体查询能够检测视口的宽度，然后编写差异化的 CSS 样式

当某个条件成立, 执行对应的CSS样式




```
@media (width:320px) {
  html {
    background-color: green;
  }
}
```

```
<title>媒体查询</title>
  <style>
    /* 视口宽度是320，网页背景色是红色 */
    @media (width: 375.33px) {

      body {
        background-color: red;
      }
    }
  </style>
</head>
<body>

</body>
```

rem 布局

目前rem布局方案中，将网页等分成10份， HTML标签的字号为视口宽度的 1/10。



```
/* 视口宽度320px，根字号为32px */
@media (width: 320px) {
    html {
        font-size: 32px;
    }
}


/* 视口宽度375px，根字号为37.5px */
@media (width: 375px) {
    html {
        font-size: 37.5px;
    }
}

/* 视口宽度414px，根字号为41.4px */
@media (width: 414px) {
    html {
        font-size: 41.4px;
    }
}
```

flexible.js

flexible.js 是手淘开发出的一个用来适配移动端的 js 库。

核心原理就是根据不同的视口宽度给网页中 html 根节点设置不同的 font-size。

```
<body>
  ......
  <script src="./js/flexible.js"></script>
</body>
```

例如：

```
<title>适配</title>
  <style>
    * {
      margin: 0;
      padding: 0;
    }

    /* 1.使用媒体查询，给不同视口设置不同的HTML字号 */
    /* @media (width: 375.33px) {
      html {
        font-size: 37.533px;
      }
    }

    @media (width: 320px) {
      html {
        font-size: 32px;
      }
    }

    @media (width: 414px) {
      html {
        font-size: 41.4px;
      }
    } */

    /* 2.使用rem单位设置元素的尺寸 */
    .box {
      width: 5rem;
      height: 3rem;
      background-color: pink;

    }
  </style>
</head>

<body>
  <div class="box"></div>
</body>

<script src="./js/flexible.js">
</script>

</html>
```

rem 移动适配

rem单位尺寸

1. 确定基准根字号

- 查看设计稿宽度 → 确定参考设备宽度(视口宽度) → 确定基准根字号（1/10视口宽度）

2. rem单位的尺寸

- **rem单位的尺寸 = px单位数值 / 基准根字号**

```
<title>rem适配</title>
  <style>
    /* 68 * 29 */
    .box {
      width: 1.813rem;
      height: 0.773rem;
      background-color: red;

    }
  </style>
</head>

<body>
  <div class="box"></div>
</body>

<script src="./js/flexible.js">
</script>
```

```
<title>rem适配</title>
  <style>
    * {
      margin: 0;
      padding: 0;
    }

    /* 1.使用媒体查询，给不同视口设置不同的HTML字号 */
    @media (width: 375.33px) {
      html {
        font-size: 37.533px;
      }
    }

    /* 2.使用rem单位设置元素的尺寸 */
    .box {
      width: 5rem;
      height: 3rem;
      background-color: pink;

    }
  </style>
</head>

<body>
  <div class="box"></div>
</body>
```

09-less

Less是一个CSS预处理器, Less文件后缀是`.less`。扩充了 CSS 语言, 使 CSS 具备一定的逻辑性、计算能力

注意：浏览器不识别 Less 代码，目前阶段，网页要引入对应的 CSS 文件

VS Code 插件：Easy LESS，保存 less文件后自动生成对应的 CSS 文件




```
.father {
    color: red;
    width: (68/37.5rem);

    .son {
        height: (29/37.5rem);
    }
}
```

注释

- 单行注释(无法生成到.css文件)
  - 语法：// 注释内容
  - 快捷键：ctrl + /
- 块注释
  - 语法：/* 注释内容 */
  - 快捷键： Shift + Alt + A

运算

- 加、减、乘直接书写计算表达式
- 除法需要添加 小括号 或 .
- 表达式存在多个单位以第一个单位为准

```
.box {
    width: 100 + 50px;
    height: 5 * 32px;

    width: (100 / 4px);
    height: 100 ./ 4px;
}
```

```
//加减乘
.box {
  width: 100 + 20px;
  width: 100 - 80px;
  width: 100 * 2px;

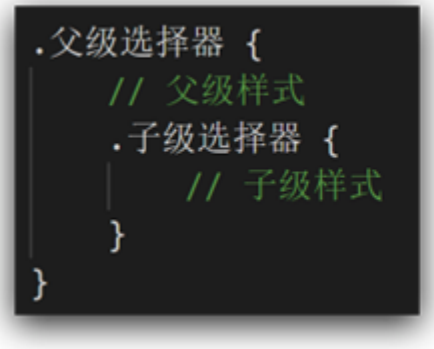

  // 除法 / → (计算表达式) 或 ./ → 推荐（）

  width: (68 / 37.5rem);
  width: 29 ./ 37.5rem;


// 如果表达式有多个单位，最终css里面以第一个单位为准
  height: (29px / 37.5rem);
}
```

嵌套

作用：快速生成后代选择器




```
.father {
  color: red;
  .son {
    width: 100px;
  }
}
```

提示：用 & 表示当前选择器，不会生成后代选择器，通常配合伪类或伪元素使用





```
.father {
  color: red;

  .son {
    width: 200px;

    a {
      color: green;

      // & 表示的是当前选择器，代码写到谁的大括号里面就表示谁 → 不会生成后代选择器
      // 应用：配合hover伪类或nth-child结构伪类使用
      &:hover {
        color: blue;
      }
    }

    // a:hover {
    //   color: orange;
    // }
```

变量

概念：容器，存储数据

作用：存储数据，方便使用和修改

语法：

- 定义变量：@变量名: 数据;
- 使用变量：CSS属性：@变量名;

```
// 1.定义变量
@myColor: pink;

// 2.使用变量
.box {
  color: @myColor;
}
a {
  color: @myColor;
}
```

导入

作用：导入 less 公共样式文件

语法：导入:@import “文件路径”;

提示：如果是 less 文件可以省略后缀

```
@import './base.less';
@import './common';
```

导出

写法：在 less 文件的第一行添加 // out: 存储URL

提示：文件夹名称后面添加 /

```
// out : ./css/mycss.css
// out: ./index.css
// out: ./css/
```

禁止导出

写法：在 less 文件第一行添加: `// out: false`

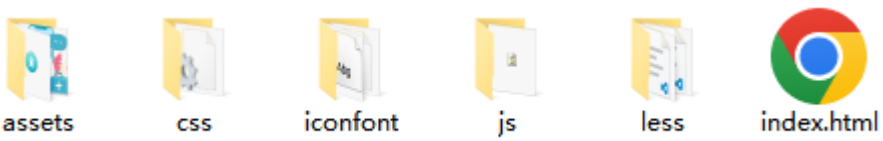


10-综合案例-极速问诊



准备工作

- 项目目录



- HTML 结构

```
<link rel="stylesheet" href="./iconfont/iconfont.css">
<link rel="stylesheet" href="./css/index.css">

<body>
  ......
  <script src="./js/flexible.js"></script>
</body>
```

- less 样式

```
// out: ../css/

@import "./base";
```

头部布局

- HTML 结构

```
<!-- 头部 -->
<header>1</header>
```

- less 样式

```
// 头部
header {
  display: flex;
  justify-content: space-between;
  padding: 0 (15 / @rootSize);
  height: (44 / @rootSize);
  background-color: pink;
  line-height: (44 / @rootSize);
}
```

头部内容

- HTML 结构

```
<a href="#" class="back"><span class="iconfont icon-left"></span></a>
<h3>极速问诊</h3>
<a href="#" class="note">问诊记录</a>
```

- less 样式

```
.icon-left {
  font-size: (22 / @rootSize);
}

h3 {
  font-size: (17 / @rootSize);
}

.note {
  font-size: (15 / @rootSize);
  color: #2CB5A5;
}
```

banner 区域

- HTML 结构

```
<!-- banner -->
<div class="banner">
  <img src="./assets/entry.png" alt="">
  <p><span>20s</span> 快速匹配专业医生</p>
</div>
```

- less 样式

```
// banner
.banner {
  margin-top: (30 / @rootSize);
  margin-bottom: (34 / @rootSize);
  text-align: center;
  img {
    margin-bottom: (18 / @rootSize);
    width: (240 / @rootSize);
    height: (206 / @rootSize);
  }

  p {
    font-size: (16 / @rootSize);
    span {
      color: #16C2A3;
    }
  }
}
```

问诊类型布局

- HTML 结构

```
<!-- 问诊类型 -->
<div class="type">
  <ul>
    <li>
      <a href="#">
        <div class="pic">1</div>
        <div class="txt">2</div>
        <span class="iconfont icon-right"></span>
      </a>
    </li>
    <li>2</li>
  </ul>
</div>
```

- less 样式

```
// 问诊类型
.type {
  padding: 0 (15 / @rootSize);
  li {
    margin-bottom: (15 / @rootSize);
    padding: 0 (15 / @rootSize);
    height: (78 / @rootSize);
    border: 1px solid #EDEDED;
    border-radius: (4 / @rootSize);
    a {
      display: flex;
      align-items: center;
      // 内容在78里面垂直居中
      height: (78 / @rootSize);
    }
  }
}
```

问诊类型内容

- HTML 结构

```
<div class="pic">
  <img src="./assets/type01.png" alt="">
</div>
<div class="txt">
  <h4>三甲图文问诊</h4>
  <p>三甲主治及以上级别医生</p>
</div>
<span class="iconfont icon-right"></span>
```

- less 样式

```
img {
  margin-right: (14 / @rootSize);
  width: (40 / @rootSize);
  height: (40 / @rootSize);
}
.txt {
  flex:1;
  h4 {
    font-size: (16 / @rootSize);
    color: #3C3E42;
    line-height: (24 / @rootSize);
  }
  p {
    font-size: (13 / @rootSize);
    color: #848484;
  }
}
.iconfont {
  font-size: (16 / @rootSize);
}
```

移动 Web 第四天

11-vw适配方案

vw和vh基本使用

vw和vh是相对单位，相对视口尺寸计算结果

- vw：viewport width（1vw = 1/100视口宽度）
- vh：lviewport height（1vh = 1/100视口高度）

```
<title>体验vw和vh</title>
    <style>
      * {
        margin: 0;
        padding: 0;
      }

      /* vw 和 vh 是相对视口宽高计算结果，可以直接实现移动端适配效果 */
      /* .box {
        width: 50vw;
        height: 30vw;
        background-color: pink;
      } */

      .box {
        width: 50vh;
        height: 30vh;
        background-color: green;
      }
    </style>
  </head>
  <body>
    <div class="box"></div>
  </body>
```

vw布局

1. 确定设计稿对应的vw尺寸（1/100视口宽度）

- 查看设计稿宽度 → 确定参考设备宽度（视口宽度） → 确定vw尺寸（1/100视口宽度）

2. vw单位的尺寸 = px 单位数值 / (1/100 视口宽度)

```
// .box {
//     width: (68/3.753vw);
//     height: (29/3.753vw);
//     background-color: #f00;
// }

.box {
    width: (68/6.67vh);
    height: (29/6.67vh);
    background-color: rgb(0, 255, 60);
}

//px单位尺寸 / 1/100视口的宽度或高度
```

```
<title>vw/vh适配</title>
    <link rel="stylesheet" href="./deno.css">
</head>

<body>
    <div class="box"></div>
</body>
```

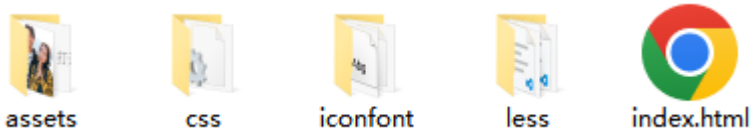
vh问题

vh是1/100视口高度，全面屏视口高度尺寸大，如果混用可能会导致盒子变形

12-综合案例-酷我音乐



准备工作



- HTML 结构

```
<link rel="stylesheet" href="./iconfont/iconfont.css">
<link rel="stylesheet" href="./css/index.css">
```

- less 样式

```
// out: ../css/

@import "./base";
```

头部布局

- HTML 结构

```
<!-- 头部 -->
<header>
  <div class="left">left</div>
  <a href="#">下载APP</a>
</header>
```

- less 样式

```
body {
  background-color: #f9fafb;
}

@vw:3.75vw;

// 头部
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 (15 / @vw);
  height: (50 / @vw);
  background-color: #fff;
}
```

头部内容

- less 样式

```
header {
  // 左边
  .left {
    width: (235 / @vw);
    height: (50 / @vw);
    background-image: url(../assets/head.png);
    background-size: contain;
    background-repeat: no-repeat;
  }
  a {
    width: (80 / @vw);
    height: (30 / @vw);
    background-color: #ffe31b;
    border-radius: (15 / @vw);
    text-align: center;
    line-height: (30 / @vw);
    font-size: (14 / @vw);
  }
}
```

搜索区域

- HTML 结构

```
<!-- 搜索 -->
<div class="search">
  <div class="txt">
    <span class="iconfont icon-sousuo"></span>
    <span>搜索你想听的歌曲</span>
  </div>
</div>
```

- less 样式

```less
// 搜索
.search {
  padding: (10 / @vw) (15 / @vw);
  height: (52 / @vw);
  // background-color: pink;
  .txt {
    height: (32 / @vw);
    background-color: #f2f4f5;
    border-radius: (16 / @vw);
    text-align: center;
    line-height: (32 / @vw);
    color: #a1a4b3;
    font-size: (14 / @vw);
    .iconfont {
      font-size: (16 / @vw);
    }
  }
}
```

banner 区域

- HTML 结构

```html
<!-- banner -->
<div class="banner">
  <ul>
    <li><a href="#"><img src="./assets/banner01.jpeg" alt=""></a></li>
  </ul>
</div>
```

- less 样式

```less
// banner
.banner {
  padding: 0 (15 / @vw);
  height: (108 / @vw);
  // background-color: pink;
  ul {
    li {
      width: (345 / @vw);
      height: (108 / @vw);
      img {
        width: 100%;
        height: 100%;
        // cover完全覆盖
        // 缩放img，图片比例跟父级盒子比例不同，避免图片挤压变形
        object-fit: cover;
        border-radius: (5 / @vw);
      }
    }
  }
}
```

标题公共样式

- HTML 结构

```html
<!-- 排行榜 -->
<div class="list">
  <!-- 标题 -->
  <div class="title">
    <h4>酷我排行榜</h4>
    <a href="#">更多<span class="iconfont icon-right"></span></a>
  </div>
</div>
```

- less 样式

```
// 排行榜
.list {
  margin-top: (20 / @vw);
  padding: 0 (15 / @vw);
}

// 标题 → 公共样式
.title {
  display: flex;
  justify-content: space-between;
  margin-bottom: (16 / @vw);
  line-height: (25 / @vw);
  h4 {
    font-size: (20 / @vw);
  }
  a {
    font-size: (12 / @vw);
    color: #a1a4b3;
  }
}
```

排行榜内容

- HTML 结构

```
<!-- 内容 -->
<div class="content">
  <ul>
    <li>
      <div class="pic"><img src="./assets/icon_rank_hot.png" alt=""></div>
      <div class="txt">
        <a href="#" class="more">酷我热歌榜<span class="iconfont icon-right"></span></a>
        <a href="#">1.樱花树下的约定（完整版） - 旺仔小乔</a>
        <a href="#">2.就让这大雨全都落下 - 容祖儿</a>
        <a href="#">3.缺氧 - 轩姨（相信光）</a>
      </div>
    </li>
    <li>
      <div class="pic"><img src="./assets/icon_rank_new.png" alt=""></div>
      <div class="txt">
        <a href="#" class="more">酷我新歌榜<span class="iconfont icon-right"></span></a>
        <a href="#">1.樱花树下的约定（完整版） - 旺仔小乔</a>
        <a href="#">2.就让这大雨全都落下 - 容祖儿</a>
        <a href="#">3.缺氧 - 轩姨（相信光）</a>
      </div>
    </li>
    <li>
      <div class="pic"><img src="./assets/icon_rank_rise.png" alt=""></div>
      <div class="txt">
        <a href="#" class="more">酷我飙升榜<span class="iconfont icon-right"></span></a>
        <a href="#">1.樱花树下的约定（完整版） - 旺仔小乔</a>
        <a href="#">2.就让这大雨全都落下 - 容祖儿</a>
        <a href="#">3.缺氧 - 轩姨（相信光）</a>
      </div>
    </li>
  </ul>
</div>
```

- less 样式

```
// 排行榜
.list {
  margin-top: (20 / @vw);
  padding: 0 (15 / @vw);
  li {
    display: flex;
    margin-bottom: (16 / @vw);
    height: (105 / @vw);
    background-color: #fff;
    border-radius: (10 / @vw);
    .pic {
      margin-right: (20 / @vw);
      img {
        width: (105 / @vw);
        height: (105 / @vw);
        border-radius: (10 / @vw);
      }
    }
    .txt {
      a {
        display: block;
        font-size: (12 / @vw);
        color: #a1a4b3;
        line-height: 1.8;
      }

      .more {
        font-size: (14 / @vw);
        color: #333;
        .iconfont {
          font-size: (16 / @vw);
        }
      }
    }
  }
}
```

推荐歌单布局

- HTML 结构

```
<!-- 推荐歌单 -->
<div class="recommend">
  <!-- 标题 -->
  <div class="title">
    <h4>推荐歌单</h4>
    <a href="#">更多<span class="iconfont icon-right"></span></a>
  </div>
  <!-- 内容 -->
  <div class="content">
    <ul>
      <li>1</li>
      <li>1</li>
      <li>1</li>
      <li>1</li>
      <li>1</li>
      <li>1</li>
    </ul>
  </div>
</div>
```

- less 样式

```
// 推荐歌单
.recommend {
  padding: 0 (15 / @vw);
  ul {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
    li {
      margin-bottom: (16 / @vw);
      width: (105 / @vw);
      height: (143 / @vw);
      background-color: pink;
    }
  }
}
```

推荐歌单内容

- HTML 结构

```
<li>
  <div class="pic">
    <img src="./assets/song01.jpeg" alt="">
    <div class="cover">18.2W</div>
  </div>
  <div class="txt">抖音嗨爆DJ！劲爆旋律萦绕双耳</div>
</li>
<li>
  <div class="pic">
    <img src="./assets/song02.jpeg" alt="">
    <div class="cover">18.2W</div>
  </div>
  <div class="txt">抖音嗨爆DJ！劲爆旋律萦绕双耳</div>
</li>
<li>
  <div class="pic">
    <img src="./assets/song03.jpeg" alt="">
    <div class="cover">18.2W</div>
  </div>
  <div class="txt">抖音嗨爆DJ！劲爆旋律萦绕双耳</div>
</li>
<li>
  <div class="pic">
    <img src="./assets/song02.jpeg" alt="">
    <div class="cover">18.2W</div>
  </div>
  <div class="txt">抖音嗨爆DJ！劲爆旋律萦绕双耳</div>
</li>
<li>
  <div class="pic">
    <img src="./assets/song03.jpeg" alt="">
    <div class="cover">18.2W</div>
  </div>
  <div class="txt">抖音嗨爆DJ！劲爆旋律萦绕双耳</div>
</li>
<li>
  <div class="pic">
    <img src="./assets/song01.jpeg" alt="">
    <div class="cover">18.2W</div>
  </div>
  <div class="txt">抖音嗨爆DJ！劲爆旋律萦绕双耳</div>
</li>
```

- less 样式

```less
// 图片
.pic {
  position: relative;
  width: (105 / @vw);
  height: (105 / @vw);
  img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    border-radius: (10 / @vw);
  }
  .cover {
    position: absolute;
    left: 0;
    bottom: 0;
    width: (70 / @vw);
    height: (28 / @vw);
    background-color: rgba(0,0,0,0.8);
    border-radius: 0 (10 / @vw) 0 (10 / @vw);
    text-align: center;
    line-height: (28 / @vw);
    color: #fff;
    font-size: (14 / @vw);
  }
}
// 文字
.txt {
  font-size: (14 / @vw);
}
```

下载区域

- HTML 结构

```html
<!-- 安装，下载 -->
<div class="download">
  <img src="./assets/logo.png" alt="">
  <p>安装酷我音乐 发现更多好音乐</p>
  <span class="iconfont icon-right"></span>
</div>
```

- less 样式

```less
// 下载
.download {
  position: fixed;
  left: (15 / @vw);
  bottom: (30 / @vw);

  display: flex;
  align-items: center;
  padding: 0 (10 / @vw) 0 (15 / @vw);
  width: (345 / @vw);
  height: (45 / @vw);
  background-color: #fff;
  border-radius: (22 / @vw);
  img {
    margin-right: (10 / @vw);
    width: (36 / @vw);
    height: (36 / @vw);
  }
  p {
    flex: 1;
    font-size: (14 / @vw);
  }
  span {
    width: (32 / @vw);
    height: (32 / @vw);
    background-color: #f2f3f5;
    border-radius: 50%;
    text-align: center;
    line-height: (32 / @vw);
    font-size: (16 / @vw);
  }
}
```

头部固定

- less 样式

```
// 头部
header {
  position: fixed;
  left: 0;
  top: 0;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 (15 / @vw);
  // 固定定位的盒子，宽度靠内容撑开，希望宽度100%
  width: 100%;
  height: (50 / @vw);
  background-color: #fff;
}

// 搜索
.search {
  // 头部固定定位，脱标不占位，搜索去最顶了，加上外边距挤下来即可
  margin-top: (50 / @vw);
  padding: (10 / @vw) (15 / @vw);
  height: (52 / @vw);
}
```

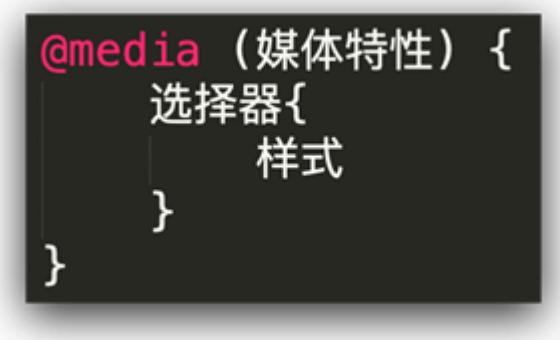
移动 Web 第五天

响应式布局方案

- 媒体查询
- Bootstrap框架

13-媒体查询

基本写法



- max-width：最大宽度（小于等于）
- min-width：最小宽度（大于等于）

```
<title>媒体查询</title>
    <style>
      /* 屏幕宽度小于等于768，网页背景色是粉色 → max-width */
      @media (max-width: 768px) {
        body {
          background-color: pink;
        }
      }


      /* 屏幕宽度大于等于1200，网页背景色是绿色 → min-width */
      @media (min-width:1200px) {
        body {
          background-color: green;
        }
      }

    </style>
  </head>
  <body>

  </body>
```

书写顺序

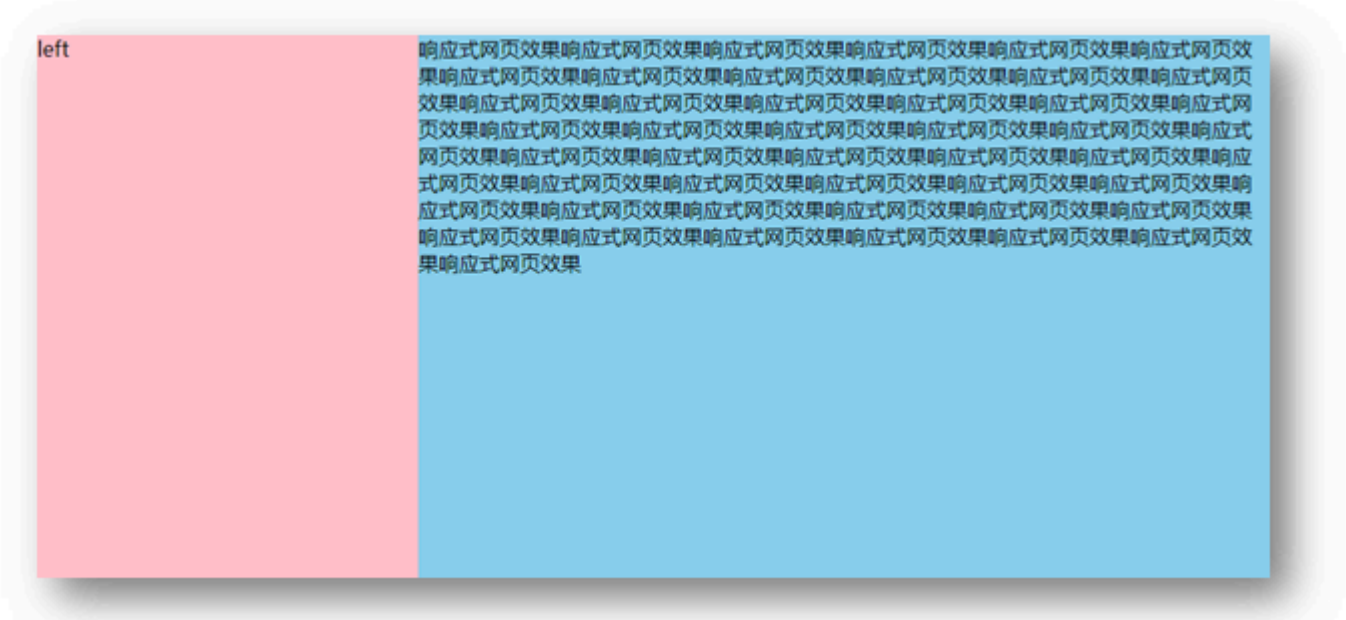
- min-width（从小到大）
- max-width（从大到小）
- 不能乱序，CSS有层叠性

```
<title>Document</title>
    <style>
      /* 网页默认背景色是灰色 */
      body {
        background-color: #ccc;
      }
      /* 屏幕宽度 大于等于 768px，网页背景色是粉色 min */
      @media (min-width: 768px) {
        body {
          background-color: pink;
        }
      }
      /* 屏幕宽度 大于等于 992px，网页背景色是绿色 min */
      @media (min-width:992px) {
        body {
          background-color: green;
        }
      }

      /* 屏幕宽度 大于等于 1200px，网页背景色是 skyblue  min */
      @media (min-width: 1200px) {
        body {
          background-color: skyblue;
        }
      }
    </style>
  </head>
  <body></body>
```

案例-左侧隐藏

需求：网页宽度小于等于768px则隐藏左侧区域


left
响应式网页效果响应式网页效果响应式网页效果

```css
.box {
  display: flex;
}

.left {
  width: 300px;
  height: 500px;
  background-color: pink;
}

.main {
  flex: 1;
  height: 500px;
  background-color: skyblue;
}

@media (max-width: 768px) {
  .left {
    display: none;
  }
}
```

媒体查询-完整写法



关键词 / 逻辑操作符

- and
- only
- not

媒体类型

媒体类型用来区分设备类型

- screen：屏幕设备
- 打印预览：print
- 阅读器：speech
- 不区分类型：all

媒体特性

- 视口宽高：width / height
- 视口最大宽高：max-width ； max-height
- 视口最小宽高：min-width；min-height
- 屏幕方向：orientation
  - protrait：竖屏
  - landscape：横屏

媒体查询-外部CSS

```
<link rel="stylesheet" media="逻辑符 媒体类型 and (媒体特性)" href="xx.css">
```

```
<title>媒体查询-link</title>
    <!-- 视口宽度 小于等于 768px， 网页背景色是粉色 -->
    <link rel="stylesheet" media="(max-width: 768px)" href="./pink.css">
    <!-- 视口宽度 大于等于 1200px，网页背景色是绿色 -->
    <link rel="stylesheet" media="(min-width: 1200px)" href="./green.css">
  </head>
  <body>

  </body>
```

14-Bootstrap

简介

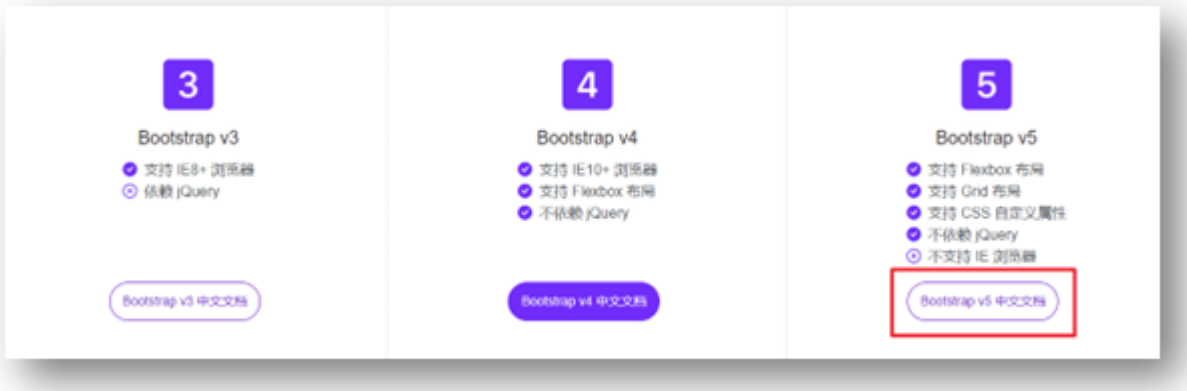
Bootstrap 是由 Twitter 公司开发维护的前端 UI 框架，它提供了大量编写好的 CSS 样式，允许开发者结合一定 HTML 结构及JavaScript，快速编写功能完善的网页及常见交互效果。

中文官网: https://www.bootcss.com/

使用步骤

下载

下载：Bootstrap V5中文文档 → 进入中文文档 → 下载 →下载 Bootstrap 生产文件



使用

1. 引入 Bootstrap CSS 文件

```
<link rel="stylesheet" href="./Bootstrap/css/bootstrap.min.css">
```

2. 调用类名： container 响应式布局版心类

```
<div class="container">测试</div>
```

栅格系统

作用：响应式布局

栅格化是指将整个网页的宽度分成12等份，每个盒子占用的对应的份数

例如：一行排4个盒子，则每个盒子占 3份 即可（12 / 4 = 3）

Breakpoint	Class infix	Dimensions
Extra small	*None*	<576px
Small	`sm`	≥576px
Medium	`md`	≥768px
Large	`lg`	≥992px
Extra large	`xl`	≥1200px
Extra extra large	`xxl`	≥1400px

	xs <576px	sm ≥576px	md ≥768px	lg ≥992px	xl ≥1200px	xxl ≥1400px
Container max-width	None (auto)	540px	720px	960px	1140px	1320px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-	.col-xxl-

- `row` 类：行，可以让内容在一行排（flex布局）
- `col-*-*`:`列（例如：col-xxl-3）

```
<title>栅格系统</title>
  <link rel="stylesheet" href="./Bootstrap/css/bootstrap.min.css">
</head>
<body>
  <!--
    视口宽度大于等于1200px，一行排4个盒子 → 3
    视口宽度大于等于768px，一行排2个盒子
    视口宽度大于等于576px，一行排1个盒子
   -->
   <!-- 版心 → row → 子级 -->
   <div class="container">
    <div class="row">
      <div class="col-xl-3 col-md-6 col-sm-12">1</div>
      <div class="col-xl-3 col-md-6 col-sm-12">2</div>
      <div class="col-xl-3 col-md-6 col-sm-12">3</div>
      <div class="col-xl-3 col-md-6 col-sm-12">4</div>
    </div>
   </div>
</body>
```

全局样式

按钮



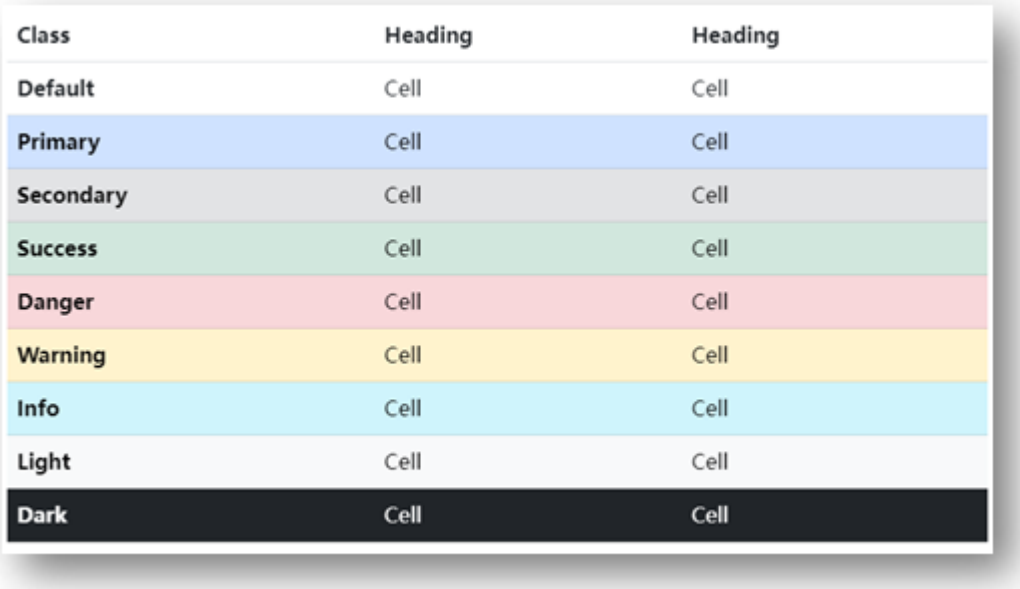
类名

- btn：默认样式
- btn-success：成功

- btn-warning：警告
- ……
- 按钮尺寸：btn-lg / btn-sm

```
<title>按钮样式</title>
  <link rel="stylesheet" href="./Bootstrap/css/bootstrap.min.css">
</head>
<body>
  <button class="btn btn-success btn-sm">按钮1</button>
  <button class="btn btn-warning btn-lg">按钮2</button>
</body>
```

表格



Class	Heading	Heading
Default	Cell	Cell
Primary	Cell	Cell
Secondary	Cell	Cell
Success	Cell	Cell
Danger	Cell	Cell
Warning	Cell	Cell
Info	Cell	Cell
Light	Cell	Cell
Dark	Cell	Cell

表格类：

- table：默认样式
- table-striped：隔行变色
- table-success：表格颜色
- ……

```
<title>表格样式</title>
    <link rel="stylesheet" href="./Bootstrap/css/bootstrap.min.css">
  </head>
  <body>
    <table class="table table-striped table-hover">
      <tr class="table-success">
        <th>姓名</th>
        <th>年龄</th>
        <th>性别</th>
      </tr>
      <tr>
        <td>张三</td>
        <td>18</td>
        <td>男</td>
      </tr>
      <tr>
        <td>李四</td>
        <td>20</td>
        <td>女</td>
      </tr>
      <tr>
        <td>王五</td>
        <td>18</td>
        <td>男</td>
      </tr>
      <tr>
        <td>赵六</td>
        <td>20</td>
        <td>女</td>
      </tr>
    </table>
  </body>
```

组件(Components)

1.引入样式表

2.引入 js 文件

3.复制结构，修改内容

```
<title>组件</title>
  <link rel="stylesheet" href="/Bootstrap/css/bootstrap.min.css">

  <style>
    .bg-body-tertiary {
      background-color: pink !important;
    }
  </style>

</head>

<body>
  <!-- 导航 -->
  <nav class="navbar navbar-expand-lg bg-body-tertiary">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">Navbar</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#n
        aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link active" aria-current="page" href="#">首页</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">课程</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">课程资料</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">视频</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- 轮播图 -->
  <div id="carouselExampleIndicators" class="carousel slide">
    <div class="carousel-indicators">
      <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="0" cl
        aria-current="true" aria-label="Slide 1"></button>
      <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="1"
        aria-label="Slide 2"></button>
```

```
      <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="2"
        aria-label="Slide 3"></button>
      <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="3"
        aria-label="Slide 4"></button>
    </div>
    <div class="carousel-inner">
      <div class="carousel-item active">
        <img src="./images/banner_1.jpg" class="d-block w-100" alt="...">
      </div>
      <div class="carousel-item">
        <img src="./images/banner_2.jpg" class="d-block w-100" alt="...">
      </div>
      <div class="carousel-item">
        <img src="./images/banner_3.jpg" class="d-block w-100" alt="...">
      </div>
      <div class="carousel-item">
        <img src="./images/banner_4.jpg" class="d-block w-100" alt="...">
      </div>
    </div>
    <button class="carousel-control-prev" type="button" data-bs-target="#carouselExampleIndicat
      data-bs-slide="prev">
      <span class="carousel-control-prev-icon" aria-hidden="true"></span>
      <span class="visually-hidden">Previous</span>
    </button>
    <button class="carousel-control-next" type="button" data-bs-target="#carouselExampleIndicat
      data-bs-slide="next">
      <span class="carousel-control-next-icon" aria-hidden="true"></span>
      <span class="visually-hidden">Next</span>
    </button>
  </div>

  <script src="/Bootstrap/js/bootstrap.min.js"></script>
</body>
```

字体图标

下载

导航 / Extend：图标库 → 安装 → 下载安装包 → bootstrap-icons-1.X.X.zip

使用

1. 复制 fonts 文件夹到项目目录
2. 网页引入 bootstrap-icons.css 文件
3. 调用 CSS 类名（图标对应的类名）

```
<i class="bi-android2"></i>
```

例如：

```
<title>字体图标</title>
  <link rel="stylesheet" href="/bootstrap-icons-1.13.1/bootstrap-icons-1.13.1/bootstrap-icons.c

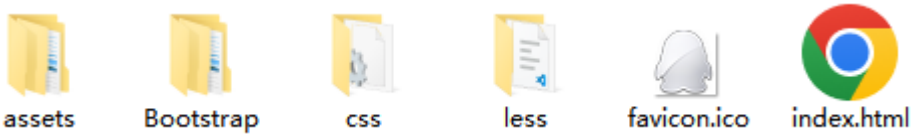
  <style>
    .bi-android2 {
      font-size: 100px;
      color: green;
    }

    .bi-alipay {
      font-size: 100px;
      color: blue;
    }
  </style>
</head>

<body>
  <span class="bi-android2"></span>
  <span class="bi-alipay"></span>
</body>
```

15-综合案例-alloyTeam

准备工作



- HTML 结构

```
<title>腾讯全端</title>
<link rel="shortcut icon" href="favicon.ico" type="image/x-icon">
<!-- 层叠性：咱们的css 要 层叠 框架的 -->
<link rel="stylesheet" href="./Bootstrap/css/bootstrap.min.css">
<link rel="stylesheet" href="./Bootstrap/font/bootstrap-icons.css">
<link rel="stylesheet" href="./css/index.css">



<script src="./Bootstrap/js/bootstrap.min.js"></script>
```

- less 样式

```
// out: ../css/
```

头部导航

- HTML 结构

```
<!-- 导航 -->
<nav class="navbar navbar-expand-lg bg-body-tertiary fixed-top">
  <div class="container">
    <a class="navbar-brand" href="#"><img src="./assets/images/logo.png" alt=""></a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#nav
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link active" aria-current="page" href="#">首页</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">博客</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Github</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">TWeb Conf</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">SuperStar</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Web前端导航</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">关于</a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

- less 样式

```scss
// 导航
.bg-body-tertiary {
  background-color: transparent !important;
  .navbar-collapse {
    flex-grow: 0;
  }
}
```

轮播图图片

- HTML 结构

```html
<!-- 轮播图 -->
<div id="carouselExampleIndicators" class="carousel slide">
  <div class="carousel-indicators">
    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="0" clas
    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="1" aria
    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="2" aria
    <button type="button" data-bs-target="#carouselExampleIndicators" data-bs-slide-to="3" aria
  </div>
  <div class="carousel-inner">
    <div class="carousel-item active">

    </div>
    <div class="carousel-item">

    </div>
    <div class="carousel-item">

    </div>
    <div class="carousel-item">

    </div>
  </div>
  <button class="carousel-control-prev" type="button" data-bs-target="#carouselExampleIndicator
    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Previous</span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-target="#carouselExampleIndicator
    <span class="carousel-control-next-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Next</span>
  </button>
</div>
```

- less 样式

```
.carousel {
  .carousel-item {
    height: 500px;
    background-size: cover;
    background-position: center 0;
  }
  .carousel-item:nth-child(1) {
    background-image: url(../assets/uploads/banner_1.jpg);
  }
  .carousel-item:nth-child(2) {
    background-image: url(../assets/uploads/banner_2.jpg);
  }
  .carousel-item:nth-child(3) {
    background-image: url(../assets/uploads/banner_3.jpg);
  }
  .carousel-item:nth-child(4) {
    background-image: url(../assets/uploads/banner_4.jpg);
  }
}
```

轮播图响应式

- less 样式

```
// 轮播图：小于768图片高度250 max；大于等于768图片高度400 min；大于等于992图片高度500
.carousel {
  // 响应式 → 媒体查询
  @media (max-width: 768px) {
    .carousel-item {
      height: 250px;
    }
  }

  @media (min-width: 768px) {
    .carousel-item {
      height: 400px;
    }
  }
  @media (min-width: 992px) {
    .carousel-item {
      height: 500px;
    }
  }


  .carousel-item {
    // height: 500px;
    background-size: cover;
    background-position: center 0;
  }
}
```

开源项目响应式

- HTML 结构

```
<!-- 开源项目 -->
<div class="project container">
  <div class="title">
    <h2>OpenSource / 开源项目</h2>
    <p>种类众多的开源项目，让你爱不释手</p>
  </div>
  <div class="content">
    <div class="row">
      <div class="col-lg-3 col-md-6">1</div>
      <div class="col-lg-3 col-md-6">2</div>
      <div class="col-lg-3 col-md-6">3</div>
      <div class="col-lg-3 col-md-6">4</div>
    </div>
  </div>
</div>
```

- less 样式

```
// 开源项目
// 视口宽度大于992；一行排4个盒子  col-lg-3
// 视口宽度大于768；一行排2个盒子  col-md-6
.project {
  margin-top: 60px;
  text-align: center;
}
```

开源项目内容布局

- HTML 结构

```
<!-- 开源项目 -->
<div class="project container">
  <div class="title">
    <h2>OpenSource / 开源项目</h2>
    <p>种类众多的开源项目，让你爱不释手</p>
  </div>
  <div class="content">
    <div class="row">
      <div class="col-lg-3 col-md-6"><a href="#">1</a></div>
      <div class="col-lg-3 col-md-6"><a href="#">1</a></div>
      <div class="col-lg-3 col-md-6"><a href="#">1</a></div>
      <div class="col-lg-3 col-md-6"><a href="#">1</a></div>
    </div>
  </div>
</div>
```

- less 样式

```scss
.project {
  margin-top: 60px;
  text-align: center;
  .row {
    div {
      margin-bottom: 10px;
      height: 200px;
      // background-color: pink;
      a {
        // 块级的宽度和父级一样大
        display: block;
        height: 200px;
        background-color: green;
        border-radius: 4px;
      }

      &:nth-child(1) a {
        background-color: #70c3ff;
      }
      &:nth-child(2) a {
        background-color: #fd6a7f;
      }
      &:nth-child(3) a {
        background-color: #7f8ea0;
      }
      &:nth-child(4) a {
        background-color: #89d04f;
      }
    }
  }
}
```

九、文字控制属性

控制文字的属性名

1. `font-size`：字体大小，单位`px`

2. `font-weight`:字体粗细

属性值：
a.数字（开发使用） → 正常：`100`，加粗：`700`
b.关键字 → 正常：`normal`，加粗：`bold`

3. `font-style`：字体倾斜

作用：清除文字默认的倾斜效果
属性值：
a.正常（不倾斜） → `normal`
b.倾斜：`italic`

4. `line-height`:行高

作用：设置多行文本的间距
属性值：
a.数字 + px
b.数字（当前标签`font-size`属性值的倍数）
行高的测量方法：从一行文字的最顶端（最底端）量到下一行文字的最顶端（最底端）。
行高-垂直居中：行高属性值 = 盒子高度属性值。只适用于单行文字

```
<style>
    div{
        height:100px;
        background-color:skyblue;

        line-height:100px;
        }
</style>

<body>
    <div>文字</div>
</body>
```

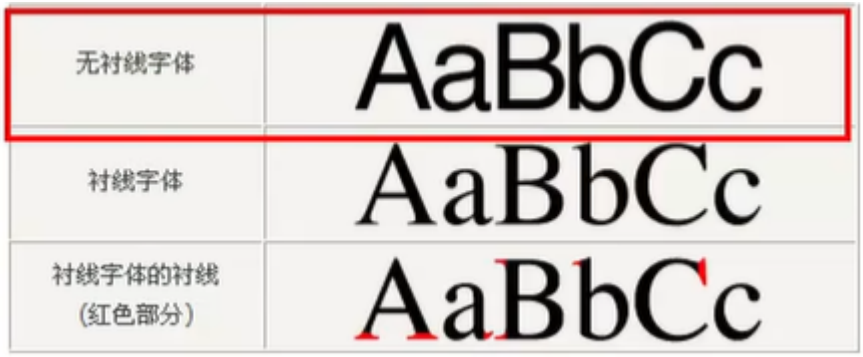
5. `font-family`:字体族，如宋体

属性值：字体名（如：宋体、楷体、黑体...）

`font-family: Microsoft YaHei,Heiti SC,tahoma,arial,Hiragino Sans GB,"\5B8B\4F53",sans-serif;`

拓展：`font-family` 属性值可以写多个字体名，各个字体名用逗号隔开，执行顺序是从左向右依次查找

· `font-family` 属性最后设置一个字体族名，网页开发建议使用无衬线字体 `sans-serif` 。



6. `font`:字体复合属性

使用场景：设置网页文字公共样式

复合属性：属性的简写方式，一个属性对应多个值的写法，各个属性值之间用空格隔开。

`font`：是否倾斜 是否加粗 字号/行高/字体（必须按顺序写）

`div{font:italic 700 90px/2 楷体;}`

注意：字号和字体值必须书写，否则 `font` 属性不生效

```
<style>
    div{
        font:italic 700 30px/2 楷体;
        }
</style>

<body>
    <div>测试font属性</div>
</body>
```

7. `text-indent`： 文本缩进，如首行缩进

属性值：

a.数字 + px

b.数字 + em（推荐：1em = 当前标签的字号大小）

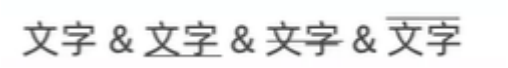
```
<style>
    p{
        text-indent:2em;
    }
</style>

<body>
    <p>这是p标签。动作：图二按照图一的动作生成一模一样的动作，左手放在下面，右手指到天空的旁边，保持人
背景：帮我生成图片：将背景换为灰色墙壁 带有横向分格线 分格线相距稍微远一点 地面换为白色边缘线的柏油路
</body>
```

8. `text-align`:文本对齐

作用：控制内容水平对齐方式，不改变标签位置
属性值：
a. `left`：左对齐（默认）
b. `center`：居中对齐
c. `right`：右对齐

9. `text-decoration`:修饰线



属性值：
a. `none`：无
b. `underline`：下划线
c. `line-through`：删除线
d. `overline`：上划线

10. `color`：颜色

属性值：
a. `颜色英文单词`:颜色关键字，red、green、blue...，学习测试用
b. `rgb(r,g,b)`：rgb表示法，r，g，b表示红绿蓝三原色，取值0~255，了解即可
c. `rgba(r,g,b,a)`：rgba表示法,r，g，b表示红绿蓝三原色，a表示透明度，取值：0~1，开发使用，实现透明色
d. `#RRGGBB`：十六进制表示法（# $0_{9,A}$F任意组合），#000000，`#ffcc00`，简写：#000，`#fc0`，开发使用（从设计稿复制）

```
<style>
    p{color:red;}
    div{
        color:rgb(0,255,0);
        }
    h1{color:rgba(0,0,0,0.3);}
    h2{color:#00ff;}
</style>

<body>
    <p>英文单词</p>
    <div>rgb</div>
    <h1>rgba</h1>
    <h2>十六进制表示法</h2>
</body>
```

十、Emmet写法

Emmet写法：代码的简写方式，输入缩写VS C ode会自动生成对应的代码。

a.HTML

说明	标签结构	Emmet
类选择器	`<div class="box"></div>`	标签名.类名
id 选择器	`<div id="box"></div>`	标签名#id名
同级标签	`<div></div><p></p>`	div+p
父子级标签	`<div><p></p></div>`	div>p
多个相同标签	`<span>1</span><span>2</span><span>3</span>`	span*3
有内容的标签	`<div>内容</div>`	div{内容}

b.CSS：大多数简写方式为属性单词的首字母

c.注释快捷方式

1.行注释（单行/多行）

Windows/Linux: `Ctrl+/`

2.块注释：

Windows+Linux： `Shift+Alt+A`

3.批量行注释（精准控制）

添加：`Ctrl+K → Ctrl+C`
移除：`Ctrl+K → Ctrl+U`